DMarlon Studio

User manual

DMarlon Studio manual

The complete guide to the visual IDE for building Python apps

Version 1.0.86 · 2026-11-09

dmarlonstudio.com

Contents

1 **Introduction and installation**
What DMarlon Studio is · Requirements · Installation · Where everything is stored

2 **Getting started**
The main window · Theme and language · Your first window in five steps

3 **The Designer**
Projects · The Designer screen · Components · Moving, aligning and editing · Window position · Events and code · Generated Python and export · Run, stop and see the output · Included examples · Data connections · HTML view: real websites · The project, its icon and the code

4 **Debugging**
Breakpoints and debugging · Stepping · Variables, stack and evaluate · History, calls, performance and objects · Tests

5 **Electron + React**
Creating an Electron project · The code: components with properties · Run, debug and export

6 **Chrome extensions**
Creating an extension · What an extension adds · Test, load in Chrome and publish

7 **Web with Next.js**
A website with screens (like Flutter) · A website with a server · A React (Vite) website · A plain HTML website · A Vue (Vite) website · A website with a Go server · Creating a Next.js page

8 **C++ with FLTK**
Creating a C++ app · Build, run and export

9 **Pascal with Lazarus**
Create a Pascal application · Compile, run and export · Example: offline Odoo and BIM projects

10 **Mobile apps with Flutter**
Create a Flutter app · Logic and bindings · Desktop · FalconDatabase and Firebase · The simulator and export

11 **Mobile apps with React Native**
Create a React Native app

12 **Console applications**
Create a console program · Examples

13 **AI in the Designer**
Chat with Claude Code · Ask the AI · AI budget · GitHub

14 **The Editor**
Opening a folder · Explorer and tabs · Find and replace · Terminal · Source control (git) · Claude: implement · AI chat

15 **Databases**
Connections · Tables and queries · FalconDatabase and Firebase

16 **API client**
Requests · Collections, variables and history

17 **FTP**
Servers and files

18 **Diagrams**
Flowcharts and database diagrams

19 **Settings**
Connect to Claude (MCP) · Artificial intelligence · Language, appearance and version

20 **Keyboard shortcuts**
Designer · Editor

21 **Troubleshooting**
Python not found, or no tkinter · Flask is missing · The app won't open (unsigned) · AttributeError: 'Form1' object has no attribute 'root' · Claude Code was not found

22 **Roadmap**
Targets · Richer projects

Chapter 1

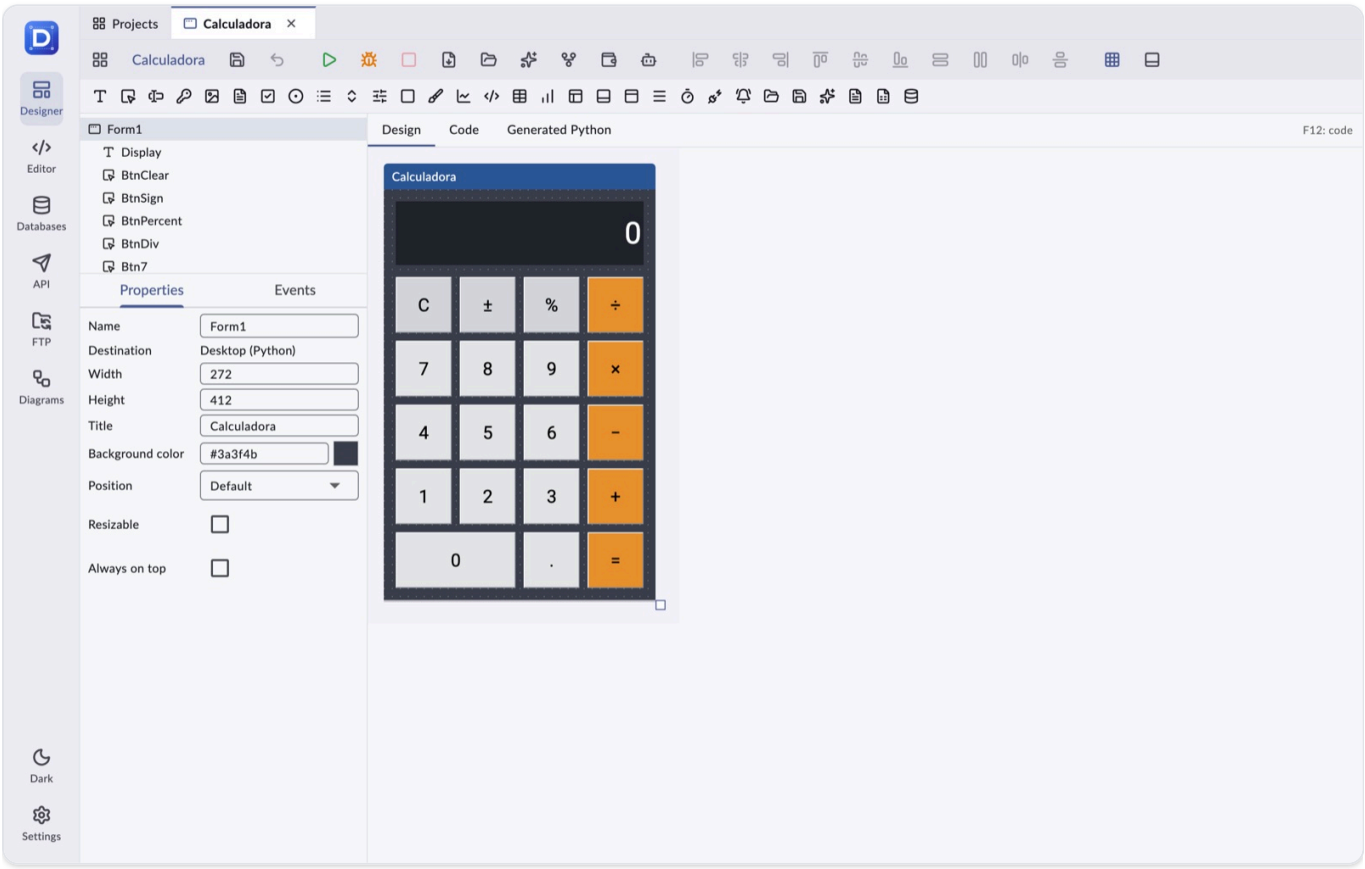
Introduction and installation

What DMarlon Studio is, what it needs and where it keeps everything.

What DMarlon Studio is

DMarlon Studio is a visual desktop IDE. You design windows by dragging components onto a form, adjust their properties in an inspector and write the code of each event in Python. **F9** runs the window on your computer.

Today the same designer builds two kinds of app: desktop windows with **tkinter** and web apps with **Flask**. Around the designer there is a VS Code-style code editor with git and a terminal, a database manager, an API client, an FTP client, a diagram editor and AI assistants (Claude by default).



The Designer: palette on top, tree and inspector on the left and the form in the center.

Requirements

To…	You need
Use the studio	macOS 10.15 or later, or 64-bit Windows 10/11.

To…	You need
Run desktop windows	Python 3 with tkinter. On macOS: `brew install python-tk` or the python.org installer. On Windows: the python.org installer with the **tcl/tk and IDLE** option ticked.
Run web apps	Flask in that same Python: `python3 -m pip install flask` (on Windows, `py -3 -m pip install flask`).
Version control in the Editor	git installed on the computer.
Claude Code chat in the Designer	Claude Code installed and signed in (optional).
AI assistants	An API key for the provider chosen in **Settings** (optional).

> The studio finds Python on its own: on the PATH, in the Homebrew folders and in the python.org install, and keeps the one with the newest Tk. The result is shown on the Designer's project list.

Installation

Download the zip for your system from the downloads section of dmarlonstudio.com.

1. **macOS**: unzip `DMarlonStudio-macOS.zip` and move **DMarlon Studio** to the Applications folder. The first time, open the app with right-click → **Open**: it is not signed.
2. **Windows**: unzip the whole `DMarlonStudio-Windows.zip` wherever you like (not in Program Files) and run `DMarlonStudio.exe`. If “Windows protected your PC” appears, click **More info** → **Run anyway**.

Where everything is stored

What	Where
Designer projects	One folder per project in `~/Documents/DMarlon Studio/Proyectos` (it can be changed in the Designer). They can be versioned with git.
Settings, connections, API requests, history and diagrams	`studio.sqlite`, in the app's data folder: `~/Library/Application Support/com.dmarlonstudio.studio` on macOS and `%APPDATA%\DMarlon Studio\DMarlon Studio` on Windows.
Deleted projects	The `.papelera` folder inside the workspace: nothing is erased for good.

Each project folder holds `project.dmsproj` (the design, as JSON), the window's code (`Form1.py`, named after the form), a `.gitignore` and, once run, `build/` with the generated Python.

Chapter 2

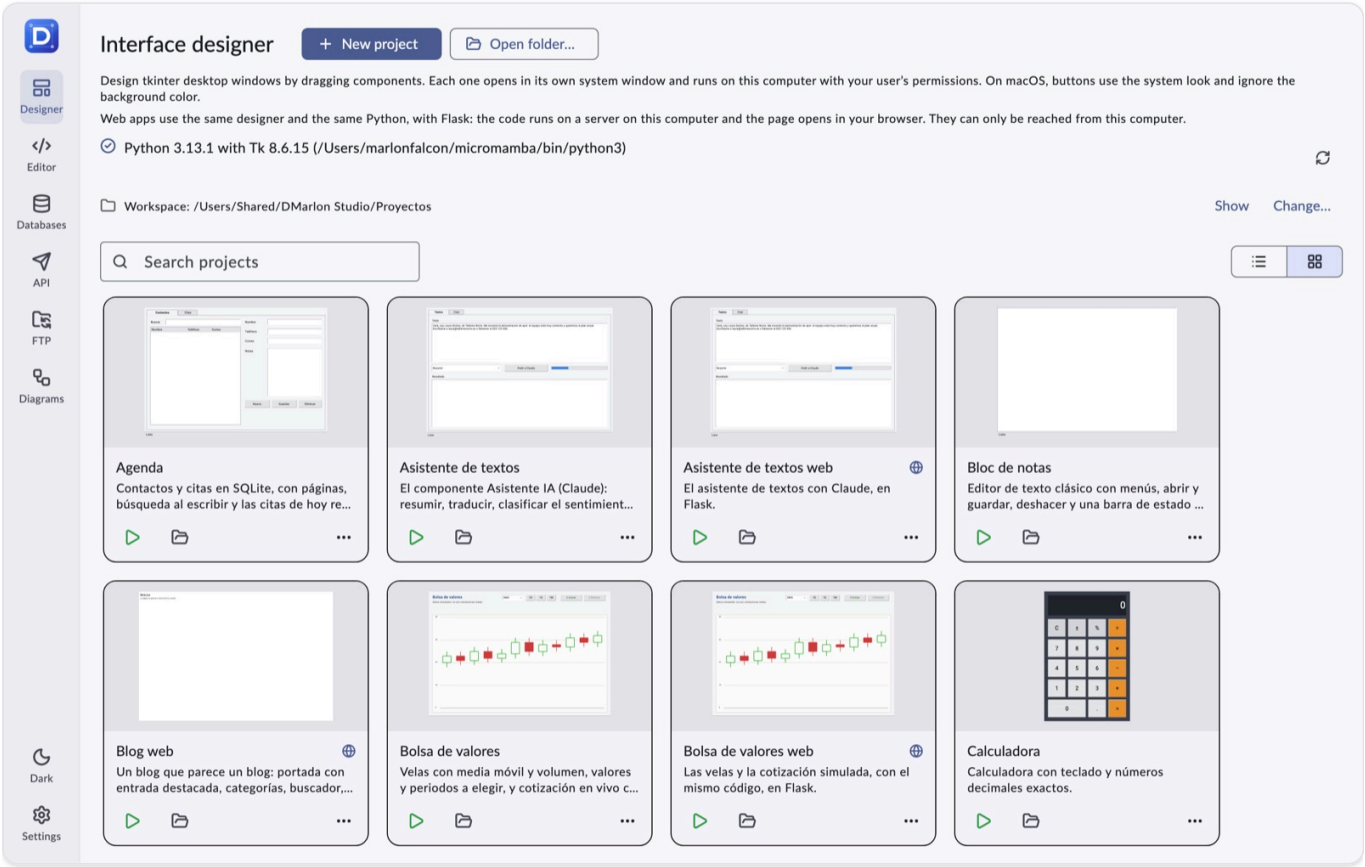
Getting started

The main window, the modules, the theme and the language.

The main window

On the left is the module bar: **Designer**, **Editor**, **Databases**, **API**, **FTP**, **Diagrams** and, at the bottom, the **Dark**/**Light** button and **Settings**.

Each module is created the first time you open it and keeps its state: you can go from the Designer to the Editor and back without losing anything unsaved.



The project list, with the examples that come with the studio.

Theme and language

- The **Dark**/**Light** button on the bar switches the theme at once. In **Settings → Appearance** you can also choose **System** to follow your computer.
- **Settings → Language** switches between **Español** and **English** without closing anything. The first time, the system language is used.

Your first window in five steps

1. In **Designer**, click **New project**, type a name and choose **Desktop (Python)**.
2. Drag a **Label**, a **Text field** and a **Button** from the palette onto the form.
3. Select the button and change its **Text** in the inspector, for example to “Greet”.
4. Double-click the button: the `Button1_click` method is created and the code opens. Type `self.Label1.config(text="Hello, " + self.Entry1.get())`.
5. Press **F9**. The window opens in its own system window.

Chapter 3

The Designer

Projects, components, inspector, events, generated code and running.

Projects

The project list shows the workspace projects and those opened from other folders. At the top you see the Python found (Python and Tk versions) and the button to **Look for Python again**.

It is shown as a **list** with columns (**Name**, **Type**, **Components**, **Modified** and **Folder**) sorted by clicking their title, or as **Cards**. It has **Search projects** and is split into pages (**Per page**: 10, 15, 25 or 50). Double-click opens a project. Right-clicking a project offers **Open**, **Run**, **Show the folder**, **GitHub…**, **AI budget**, **Duplicate** and **Delete**; right-clicking the background, **Add new** › **Desktop project (Python)**, **Web project (Flask)** or **Open folder…**.

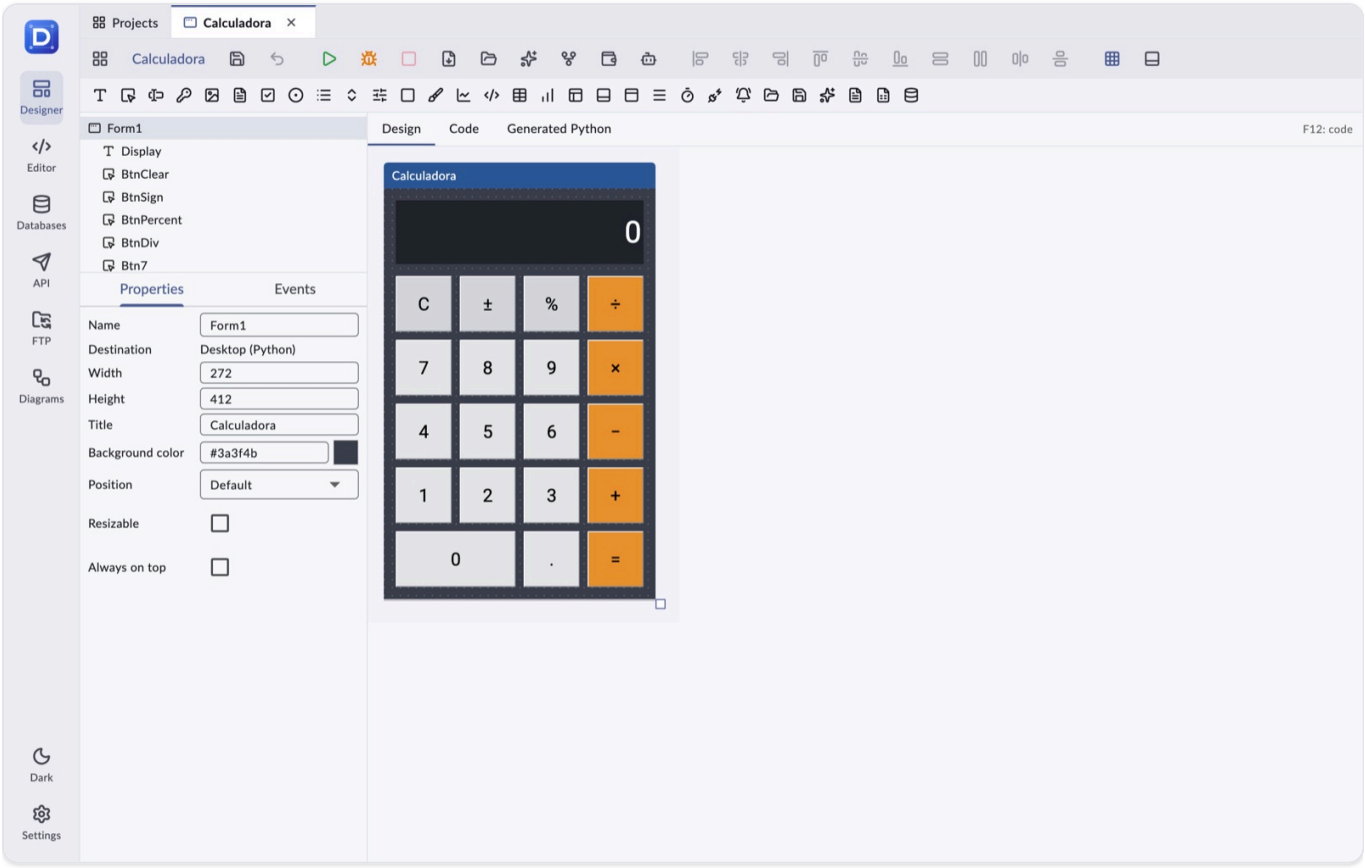
- **New project**: asks for the name and the target, **Desktop (Python)** or **Web (Flask)**. The target cannot be changed later.
- **Open folder…**: opens a project stored in any other folder; it stays on the list as a recent one.
- **Workspace**: **Show** opens it in Finder or Explorer and **Change…** picks another folder.
- On each card: **Run**, **Show the folder** and, under **More actions**, **GitHub…**, **AI budget**, **Duplicate** and **Delete** (moves the project to the workspace trash).
- **Tabs**: each open project has its tab at the top, with a dot when there are unsaved changes. Several can be open at once; the **Projects** tab goes back to the list without closing anything. Close them with the ✕ or a middle click; they ask whether to save.

> ⓘ The studio copies the example projects into the workspace (new ones also appear in existing workspaces). See the full list in **Included examples**.

The Designer screen

- **Toolbar**: back to the projects, application name (click to rename it), **Save**, **Undo**, **Run (F9)** in green, **Stop** in red, **Open in browser** (web), **Show the project folder**, **Export the .py file**, **GitHub**, **AI budget**, **Ask the AI**, **Chat with Claude Code**, alignment and centering, **Snap to grid** and **Show output**.
- **Output console**: at the bottom; drag its top edge to resize it, minimize it with the arrow, and **Copy the output** copies every message.
- **Palette**: a bar of icons; hovering shows the component's name. Drag it onto the form or add it with a double click.
- **Tree**: the window and its components, with those inside a frame or a page indented.

- **Inspector**: **Properties** and **Events** of the selected component or, if none, of the window.
- **Center**: **Design**, **Code** and **Generated Python** tabs. **F12** switches between design and code.



The example calculator in the Designer.

Components

Every component has a **Name**, a position and a size. Almost all accept **Background color**, **Text color** and font (**Font**, **Font size**, **Bold**, **Italic**), and the mouse events (**Click**, **Double click**, **Right click**, **Mouse enter**, **Mouse leave**). Typing components add the keyboard ones (**Focus in**, **Focus out**, **Key down**, **Key up**, **Enter key**).

Component	Own properties	Main events
Label (`Label`)	Text, Alignment, Justification, Relief, Line wrap	Mouse
Button (`Button`)	Text, State, Relief	**On press** (`Button1_click`)
Text field (`Entry`)	Initial text, Justification, Mask with (e.g. `*` for passwords), State, Linked table and Linked field	Keyboard and mouse
Password (`Password`)	Placeholder text, Button to show the password, Maximum length, State	Keyboard and mouse

Component	Own properties	Main events
Image (`Image`)	File (in the project's `images` folder), Fit (original size, whole without distortion, fill and crop, stretch), Border	Mouse
Long text (`Text`)	Initial text (several lines), Line wrap, State	Keyboard and mouse
Checkbox (`Checkbutton`)	Text, State, Checked	**On press**
Option (`Radiobutton`)	Text, State, Group, Value, Checked	**On press**
List (`Listbox`)	Items (one per line), Selection (browse, single, multiple, extended)	**On select**
Drop-down list (`Combobox`)	Items (one per line), Initial text, Justification, State (normal, readonly, disabled)	**On select**
Slider (`Scale`)	Title, From, Until, Step, Value, Orientation, Show value	**On change** (receives the value)
Frame (`Frame`)	Relief, Border. Holds other components	Mouse
Canvas (`Canvas`)	Background color. For drawing from code	Mouse, **Drag**, **Button release**, **Mouse move**
Data table (`Treeview`)	Data source, Query (SQL; in MongoDB, JSON), Columns (field|title|width), Limit, Sequence, Load on open	**On select**
Progress bar (`Progressbar`)	Value, Maximum, Mode (determinate or indeterminate), Orientation	Mouse
Page control (`PageControl`)	Pages (one per line), Active page. Each page holds its own components	**On page change**
Status bar (`StatusBar`)	Text. Always takes the bottom edge of the window	Mouse
Ribbon (`Ribbon`)	Tabs, groups and buttons (indented; “Button | icon”), Active tab	**On button press**, **On tab change**
Menu (`Menu`)	Options (one per line; indent = submenu; `-` = separator). Non-visual: shown at the top of the window	**On choosing an option** (receives the option)
Timer (`Timer`)	Interval (ms), Active. Non-visual	**Every interval**

Component	Own properties	Main events
API integration (`RestClient`)	Base URL, Authentication (none, Bearer token, basic or a key in a header; the secret, preferably in an environment variable), Headers, Timeout. Non-visual	**When a call finishes**, **If a call fails**
Tray icon (`TrayIcon`)	Hover text, Image, Menu (one option per line). Non-visual	**On icon click**, **On choosing a menu option**
Open dialog (`OpenDialog`)	Title, Types (“Text|*.txt”), Default extension, Initial folder, Choose several files. Non-visual	**On choosing a file**
Save dialog (`SaveDialog`)	Title, Types, Default extension, Initial folder, File name, Ask before overwriting. Non-visual	**On choosing a file**
AI assistant (`AiAssistant`)	Claude model, Instructions, Effort, Maximum answer length. Non-visual. From code: `ask` , `chat` (with memory; `reset` starts another), `summarize` , `translate` , `classify` (one label from a list), `extract` (data into a dictionary) and their `*_async` variants, which don't freeze the window. Uses Anthropic's official SDK (`pip install anthropic`); when run from the studio it gets the key from **Settings** through the `ANTHROPIC_API_KEY` variable	**When the answer arrives**, **If the request fails**
PDF report (`PdfReport`)	Document title, File name, Orientation, Author, Page numbers. Non-visual. From code: `title` , `heading` , `text` , `table` (with totals and the header repeated on every page), `image` (JPEG), `line` , `page_break` ; then `save` , `save_dialog` , `open` or `download` (web). Nothing to install	—
Excel report (`ExcelReport`)	File name, First sheet. Non-visual. From code: `sheet` , `header` , `write_row` , `write_rows` (text, numbers, dates and booleans with their type; frozen header with a filter); then `save` , `save_dialog` , `open` or `download` (web)	—
Chart (`Chart`)	Kind (line, area, bar or candles), Title, Axis titles, Legend, Grid, Decimals, Value prefix and suffix, Maximum points. From code: `set_series` , `add_point` , `set_candles` , `add_candle` , `clear`	**On clicking a point**
Data connection (`Connection`)	Connection (one of those saved in **Databases**). Non-visual	—

> Menu, Progress bar, Status bar, Timer and Page control arrive with version 1.0.5; Password, Image, Ribbon, API integration, Tray icon and the Open and Save dialogs, with 1.0.6; the Chart, with 1.0.8. In a web app, components without an equivalent say so in the inspector.

Moving, aligning and editing

- Click selects; dragging moves; the handles resize. With **Snap to grid** on, everything fits the grid.
- The arrow keys move the selection one grid step; with Alt (Option), one pixel; with Shift, they resize.
- With two or more components selected, **Align left**, **Center horizontally**, **Align right**, **Align top**, **Center vertically**, **Align bottom**, **Same width** and **Same height** become available (relative to the first one selected).
- With one or more selected, **Center in the window (horizontally)** and **Center in the window (vertically)** center them as a block in the window or in their frame.
- Right-click offers **Cut**, **Copy**, **Paste**, **Delete**, **Bring to front** and **Send to back**.
- **Copy between projects**: copied items go to the system clipboard, with their children and **the code of their events**. Paste them into another project (or after restarting the studio): components get free names and their methods are renamed with them, never overwriting an existing one. Helper methods called by those events must be copied by hand.
- **Undo** (⌘Z / Ctrl+Z) reverts each design change. If the code was edited by hand after a step, that step restores the design but not the code, and you are told so.

Window position

The window's **Position** property decides where it opens: **Centered on screen** (new projects), **Default** (the system decides) or **Manual**, with **Left** and **Top**.

Events and code

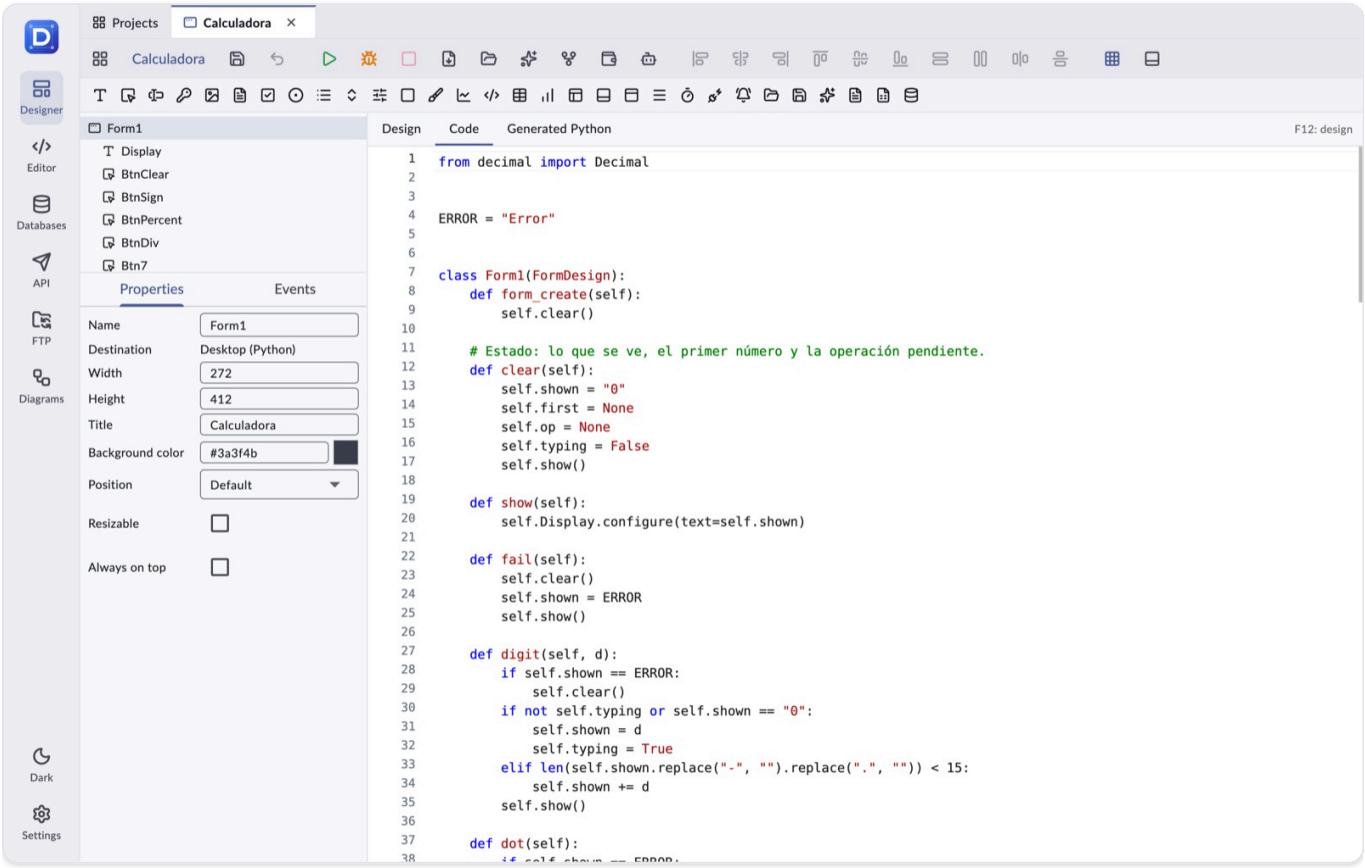
The window's code is a Python class named after the form that inherits from `FormDesign`, the class the studio generates. `self.root` is the window and each component is `self.<Name>`.

- Double-clicking a component opens its main event (`Button1_click`, `Listbox1_select`…) and creates it if needed.
- On the inspector's **Events** tab you type or choose the method for each event; **Go to code** jumps to it.
- Renaming a component renames its default methods and the references in the code.
- The window has its own events: **On open** (`form_create`), **On close**, **Key down** and **On resize**, and its properties **Title**, **Background color**, **Resizable** and **Always on top**.

```
class Form1(FormDesign):
    def form_create(self):
        self.total = 0
        self.StatusBar1.config(text="Ready")

    def Button1_click(self):
        self.total += 1
        self.Label1.config(text=f"Clicked {self.total} times")
```

⚠ Put what should happen at startup in the **On open** event (`form_create`). If you define your own `__init__` , call `super().__init__()` first.

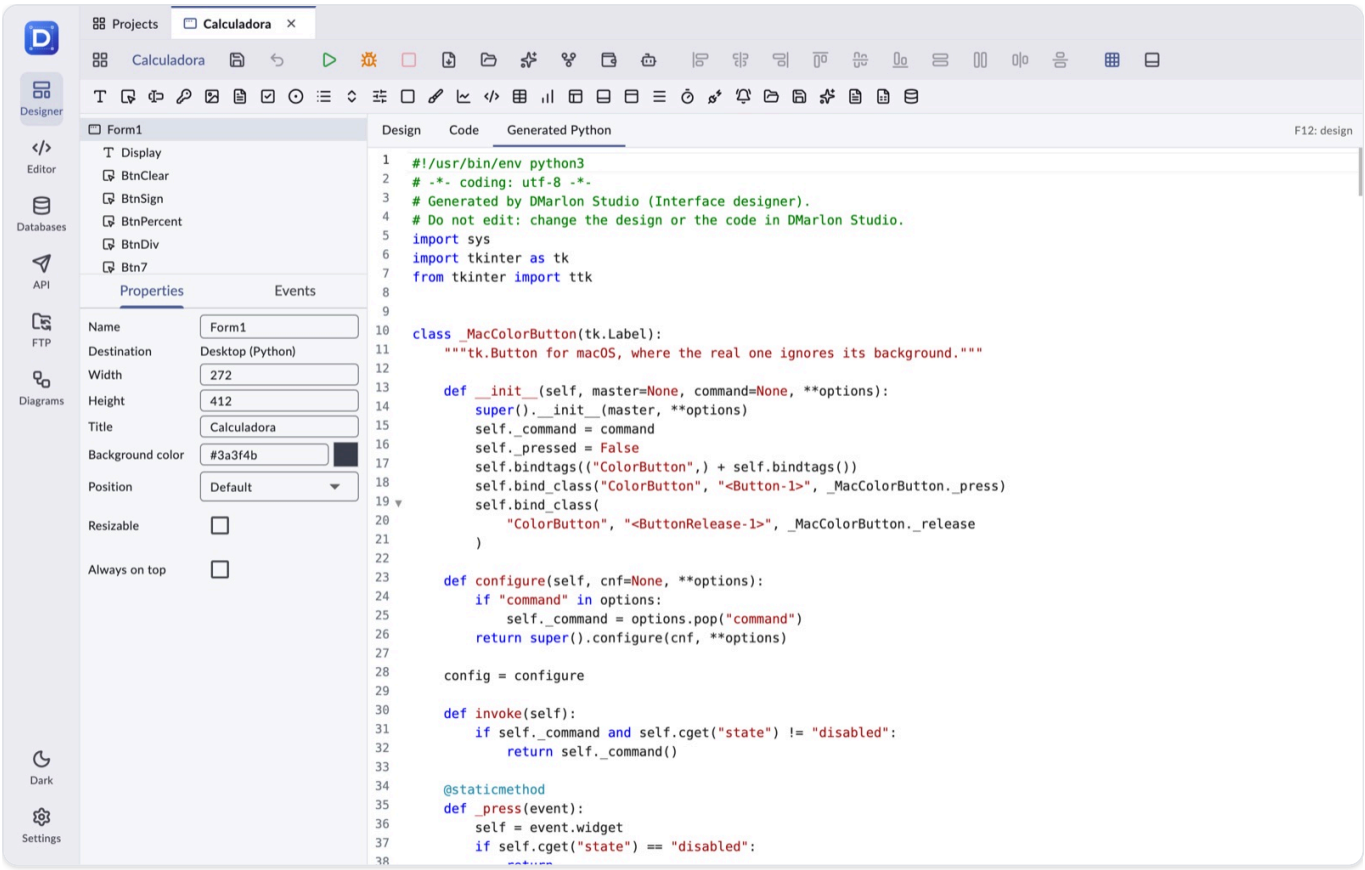


The Code tab: the window class with its handlers.

Generated Python and export

The **Generated Python** tab shows, read-only, the complete file that runs: the design turned into code plus your class. When you run, it is written to `build/<project>.py` inside the project folder.

Export the .py file saves that file wherever you want. It works with any Python 3 with tkinter (or Flask, for web apps) without the studio.



The generated Python: not edited, rebuilt from the design and the code.

Run, stop and see the output

- **Run (F9)** saves the project and launches the window in its own system window, with your user's permissions.
- **Stop** closes the running window or server.
- **Show output** opens the panel with what the app writes (`print`, Python errors) and the Python used.
- For a web app, the studio starts the Flask server on this computer and opens the browser; **Open in browser** opens it again. It can only be reached from this computer.
- If a Python package is missing (Flask, the Anthropic SDK, a database driver or pystray), the studio says so before running and offers **Install**: it installs it with the pip of the same Python that runs the project and then carries on. You can also **Copy the command** to do it by hand.

Included examples

They all keep their data in the project folder and work without any setup. Those marked (web) also exist as a Flask app.

Example	What it shows
Calculator (web)	Buttons, events and state
Unit converter (web)	Drop-down lists and computing while typing
Tasks (web)	A list with adding, removing and saving
Stopwatch (web)	Timer, menu, progress and status bars
Customer form (web)	Page control, lists and validation
Notes editor (web)	Ribbon and the Open and Save dialogs
API query (web)	Background API integration and a data table
Notepad	Classic menu, open, save, undo and a status bar with line and column
Diary	Contacts and appointments in SQLite, search and pages
Currency converter	Real exchange rates (ECB) with API integration and an offline fallback
The snake	A game on a canvas with a timer and a saved high score
Music player	Playlist, volume and progress (best with `pygame`)
Monthly sales (web)	Line, area or bar chart and a live mode
Stock market (web)	Candles, moving average and volume with live simulated data
Windows and dialogs	Messages, questions, asking for data, another window that returns a result, a modal dialog and non-modal windows
Text assistant (web)	Claude to summarize, translate, classify and extract data, and a chat with memory
Reports (web)	A sales table exported to PDF and Excel
3D viewer	3D shapes drawn on a canvas: mouse rotation, animation, lighting and wireframe mode

Example	What it shows
Email	Receive over IMAP and send over SMTP with attachments, without freezing the window
CRM web, **Invoicing web**	Complete web apps with SQLite
Web blog	A real website made with the HTML view: home page, articles, comments, newsletter, accounts with login and sign-up, and a dashboard
Web shop	Electronics shop: deals, cart, accounts, simulated payment, “My account” and a sales and stock dashboard
Odoo / ERPSqlite contacts (web)	Connecting to an Odoo over XML-RPC
PDF editor	A pip package (`pypdf`): rotate, reorder, merge, extract and split pages, text stamp, password and undo
Disk space	Background scanning without freezing the window: folder tree, largest files and totals by type
Port scanner	Networking with sockets and many parallel connections, with progress and CSV export
Stock investor (demo)	Live simulated prices, buying and selling with commission, a saved portfolio and a chart
Point of sale	A POS in the style of Odoo's with pictures: quantity, discount and price keypad, payment, receipt and sales in SQLite. It connects to Odoo with URL, user and password to download products and prices and send the orders to Sales
File renamer	Rules with regular expressions and numbering, a preview with conflicts and undo
Offline Odoo (Pascal)	A native Lazarus app for Odoo 8 to 20 over JSON-RPC: dashboard with charts, customers, vendors, products, sales and purchases end to end (quotation or request, confirmation, invoice and payment by bank or cash) and a point of sale that creates order, delivery and invoice, online and offline, without duplicates. With BIM 2.0 (base_bim_2) in Odoo, also projects and budgets by concepts (chapters, items and labour, equipment, material and subcontract resources) made offline and uploaded to Odoo
Train tracking	Live public data (Renfe's GTFS-Realtime) in the background, proximity alerts, a train graph and a map drawn on canvases, and an offline simulation
MP3 player	A custom interface drawn on canvases (digital clock, spectrum), ID3 tags, M3U lists and sound with `pygame`

Example	What it shows
File Explorer (C++)	A native C++ app with FLTK: custom components drawn in code (a tree that loads as you open it, icon, list and details views, tabs, breadcrumbs in the address bar), saved bookmarks, copy and paste with the system file clipboard (between windows and with Finder or Explorer), Trash, properties worked out in the background and a light or dark theme
Image Editor (C++)	A native image editor in C++ with FLTK: the design's Timer animates the selection; the toolbar, the zoomable canvas (sharp on Retina screens), the Color, Layers and History panels and the dialogs with live preview are drawn in code; layers with opacity and eight blend modes, a history that shares the layers that do not change, adjustments and filters, crop, resize with bilinear or bicubic resampling, rotate by any angle and its own JPEG and BMP encoders (it opens whatever FLTK reads and keeps the layers in .dmi)
Flight Finder (C++)	A native C++ app with FLTK that watches routes: the design's Timer starts a check every 30 minutes; the queries to the Travelpayouts API (Aviasales) go through the curl program on another thread and the JSON is parsed by hand; each airline's best price with its buy link, the price of each day of the month and how the best price changes are drawn in code, with system alerts when it drops; without a token, believable sample data
Hardware Inspector (C++)	A native C++ app with FLTK that analyzes the computer: sysctl and system_profiler as JSON (macOS) or the Win32 API and PowerShell with Get-CimInstance (Windows) on another thread; the design's Timer measures the processor and each core, the memory, the network, the disks and the battery every second; the cards, the charts and the tables are drawn in code, and the multi-page report goes to PDF with Fl_PDF_File_Surface or to the printer with Fl_Printer, with serial numbers hidden unless you ask for them
Motorbike Racing (C++)	A native C++ game with FLTK: the design's Timer moves the race 60 times a second; the pseudo-3D track (projected segments with bends and hills), the leaning bikes, the scenery and the dashboard are drawn in code; seven rivals that brake, pick their line and overtake, timed laps, standings with gaps, a minimap and a saved lap record
Under the Shadow of the Malecón (Pascal)	An investigation adventure in Object Pascal with Lazarus: a 360×220, 256-colour VGA screen drawn pixel by pixel in a TPaintBox (dithered scenes, portraits, icons and letters taken from the system font without smoothing), the design's Timer for the typed text and what moves, four acts with clues and items, and three endings
Double-Nine Dominoes	A table game in tkinter: the table, the tiles and the line of play that turns at the edge are drawn on a Canvas with its click and mouse events; the design's Timer makes the computer play (it remembers which numbers each player passed on); domino, blocked games and a game to 100 points

Twenty-one of these examples can be downloaded already built for macOS and Windows on the **Demos** page of dmarlonstudio.com, and three mobile apps and seven websites can be tried right there in the

browser (the mobile ones also as an Android APK). Each desktop demo is a single app in English and Spanish: change the language in its **Language** (Idioma) menu. Each demo has its page explaining how it works in three steps: the best way to see what the studio can do before installing it.

Data connections

1. Save the database in the **Databases** module (SQLite, PostgreSQL, MySQL/MariaDB or MongoDB).
2. Add a **Data connection** to the form and choose that connection in its **Connection** property.
3. Add a **Data table**, choose the connection in **Data source** and write the **Query** and the **Columns** (`field|title|width`). In MongoDB, the query is the same JSON as in the Databases module (`find`, `aggregate`, `count` or `distinct`).
4. To edit the selected row, link a **Text field** with **Linked table** and **Linked field**.

From code, `self.Connection1.query(sql, params)` returns rows and `self.Connection1.execute(sql, params)` changes data (always with `?` parameters, never by joining text). `self.Treeview1_refresh()` reloads the table.

With MongoDB, `query` and `execute` take the JSON as text or as a dictionary (`insert`, `update` and `delete` with `execute`), and each `"?"` value in the JSON takes the next parameter. It needs pymongo: if it is missing, the studio offers **Install**.

> Passwords are never written to the generated file: they reach the process in `DMSTUDIO_DB_…` environment variables. An exported file reads them from the environment.

HTML view: real websites

The **HTML view** shows HTML written by your code. With **Full page (web)** switched on, the whole page is that view: no form frame, its own CSS, like a normal website. The **Web blog** and **Web shop** examples are made this way.

- `self.Site.set_html(html, top=True)` changes the content (`top` goes back to the top of the page).
- A link or button with `data-action="post" data-arg="7"` calls the **On clicking a link or sending a form** event with an `HtmlAction`: `action.name` is `post` and `action.arg` is `7`.
- A `<form data-action="search">` sends its fields: read them with `action.get("q")` or in `action.form`.
- On the desktop, the view shows simple HTML (headings, paragraphs, lists, links and forms) in a text box; CSS styles only apply on the web.

```
import html

class Form1(FormDesign):
    def form_create(self):
        self.Site.set_html('<h1>Hello</h1><a href="#" data-
action="greet">Greet</a>')

    def Site_action(self, action):
        if action.name == "greet":
            name = html.escape(action.get("name", "world"))
            self.Site.set_html("<h1>Hello, %s</h1>" % name, top=True)
```

> ⚠ The HTML view paints HTML as is: always escape what the visitor writes with `html.escape`. The page only runs the engine's own scripts, so a `<script>` inside the HTML does not run.

The **Web blog** and **Web shop** come with user accounts: **Sign up**, **Log in** and **Log out**. Passwords are stored with PBKDF2 and a salt, never in clear text, and after five failed attempts you have to wait a minute. The first account to sign up is the administrator and sees the **Dashboard**: in the blog, posts, comments and user roles; in the shop, sales by category, orders and stock. In the shop, every customer sees their orders in **My account**.

The project, its icon and the code

The tree starts with the **Project** node; below it, the window and its components. When you choose it, the inspector shows the name, the description, the target, the folder (**Go to folder**) and the **app icon**.

- **Automatic icon**: every windowed project gets one without you doing anything. The studio picks it from its 500 icons using the words in the name and description (“Calculator”, “Online shop”, “BIM works”…), with a colour fixed for that name, and saves it to `images/app_icon.png`. If you rename the project, the icon follows.
- **Gallery…** shows the 500 icons with a search box (in English or Spanish) and 12 colours. The one you choose stays even if you rename the project; **Automatic** goes back to the studio's choice.
- **File…** opens the file dialog: the PNG is copied to the project's `images/`.
- **Generate with AI** draws the icon from a description (with the AI from Settings) and saves it as a 512 × 512 PNG. You see it before using it and can ask for another version.
- Every target uses it: the window icon (tkinter, Electron, C++), the Chrome extension icon or the Next.js web icon.
- **Subprojects**: a project can have parts with their own target (for example, “Odoo” with “db” in Python and “front” in Next.js). Create them with **Add subproject…**; they hang from the Project node and a click opens them in their tab. Each one is a folder inside the project folder.

- In the code, **Ctrl+Space** shows what you can write: after `self.`, `this.` or `Label1->`, the components and what each one has; it also appears by itself as you type. **⌘F / Ctrl+F** finds and replaces.
- **Help**: right-click a word in the code → **Help** (or **F1**) opens a panel on the right with what it is, how to write it and an example, in Python, JavaScript and C++. You can search any command, switch language, copy the example or **Insert** it at the cursor line.

Chapter 4

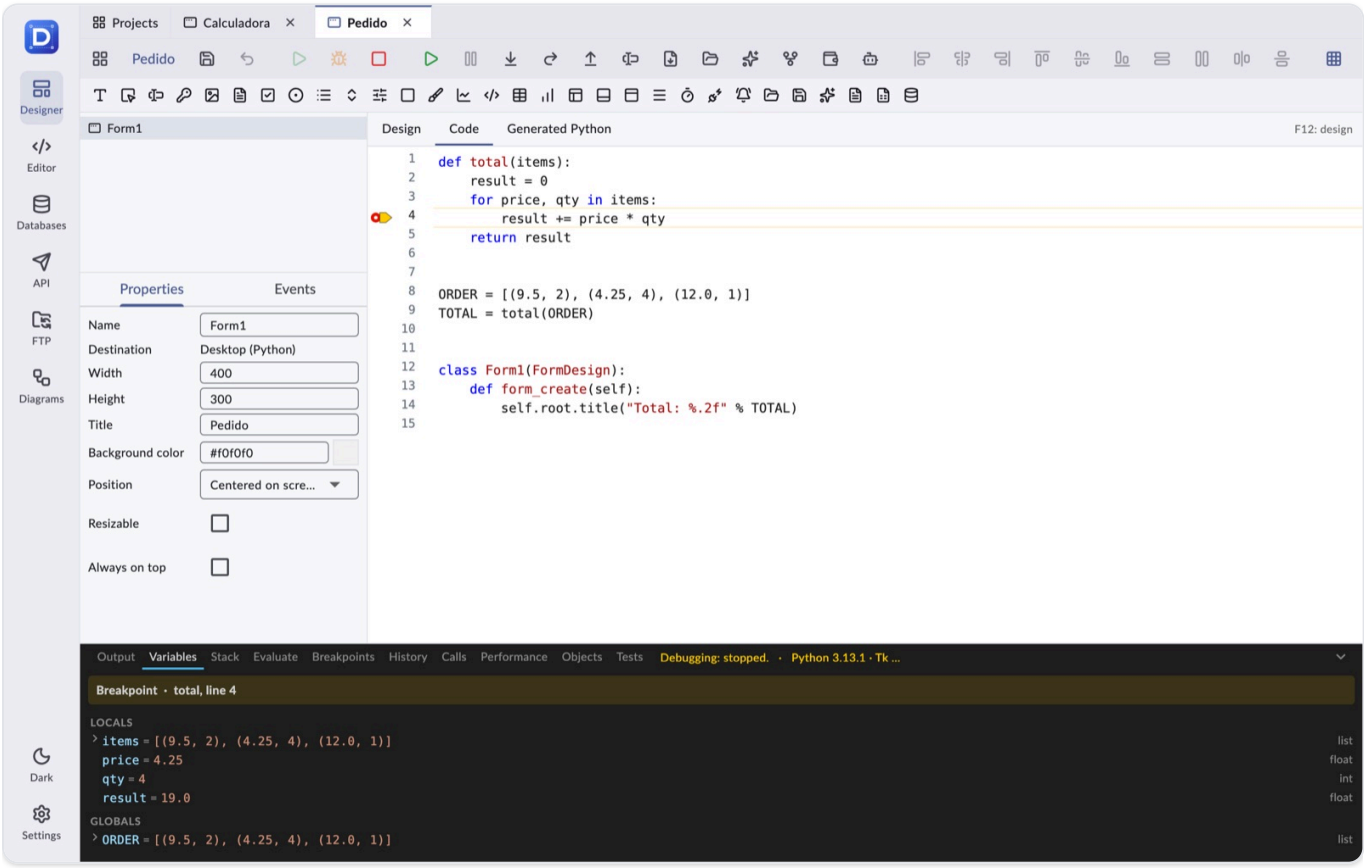
Debugging

Breakpoints, stepping, variables, evaluate, exceptions, history, performance and tests.

Breakpoints and debugging

The debugger runs your project (desktop or web) and stops it at the lines you choose in **your code**: the code generated from the design is not debugged. There is nothing to install: it uses the Python standard library.

1. In the **Code** tab, click to the left of a line number to set a **breakpoint** (a red dot); another click removes it. **F5** sets it on the cursor line.
2. **Right-click** the dot to give it a **condition** (for example `qty == 4`): it only stops when it is true. It shows a white dot inside.
3. Press **Debug** (the orange bug, **Shift+F9**). The program starts and stops when it reaches a breakpoint: the yellow arrow marks the line and the bottom panel opens **Variables**.



Stopped at line 4 when qty is 4: local and global variables, with lists and objects you can expand.

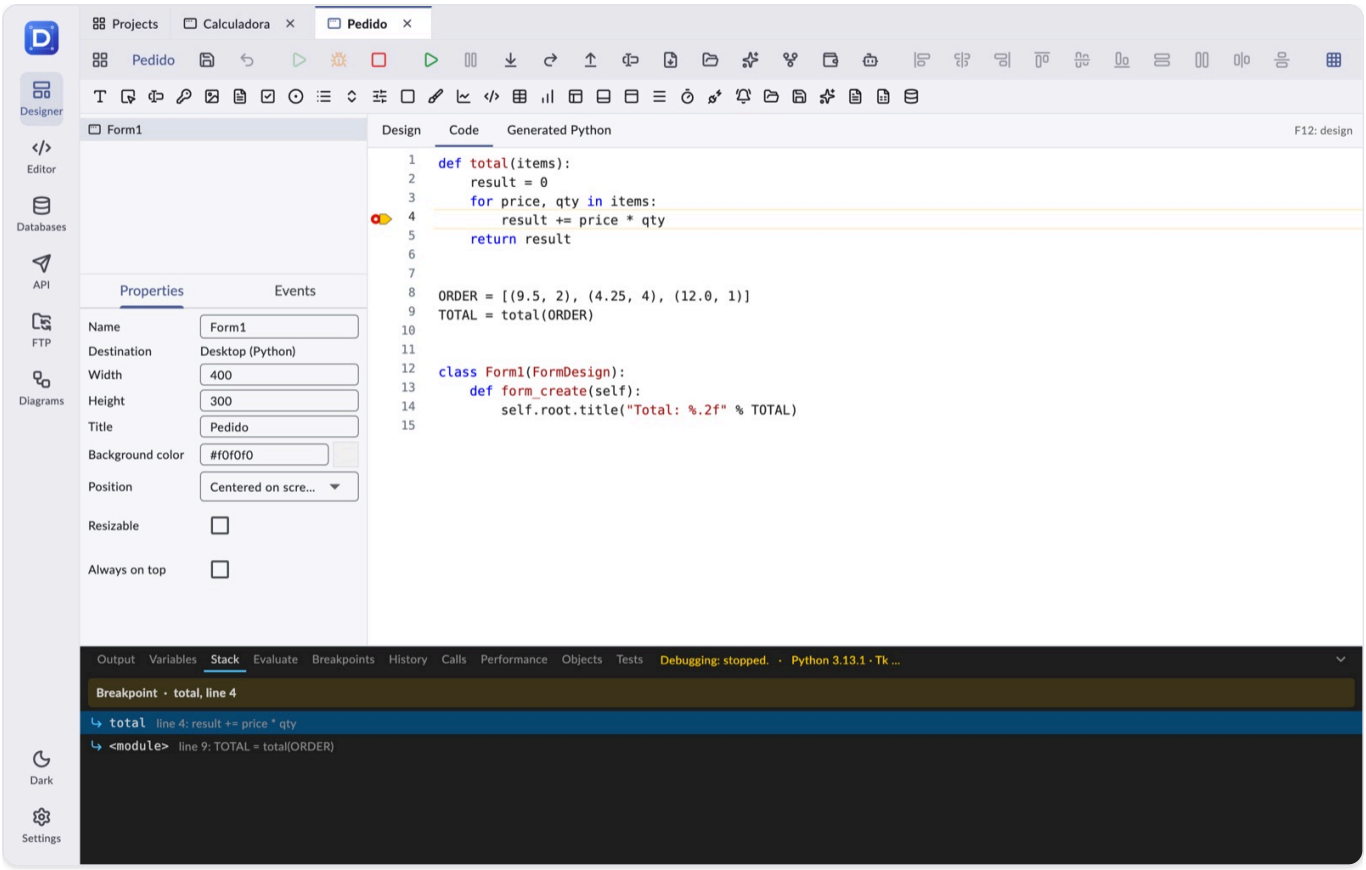
Stepping

Button	Shortcut	What it does
Continue	F9	Runs to the next breakpoint
Step into	F7	Runs the line and steps into the functions of your code it calls
Step over	F8	Runs the whole line without stepping into functions
Step out	Shift+F8	Runs until it returns to the caller of this function
Run to cursor	F4	Runs to the line where the cursor is
Pause		Stops the program at the next line of your code that runs
Stop	Ctrl+F2	Ends the program

> While it is stopped, the window (or the web page) does not respond: that is normal. When you continue, it carries on where it was.

Variables, stack and evaluate

- **Variables**: the locals and globals of the chosen frame. Lists, dictionaries and objects (such as `self`) expand to show their state.
- **Stack**: the chain of calls up to the current line. Clicking a frame chooses it: the variables change and the editor jumps to its line. If it stopped on an error, the full traceback is below.
- **Evaluate**: type a Python expression (`self.total`, `len(items)`) and press Enter. Assignments (`n = 5`) also work, to change a value before carrying on.
- **Breakpoints**: every breakpoint with its line and condition; edit or remove them here, and choose when to stop on **exceptions**: never, on uncaught ones (the default: an error in an event stops at its line) or on all of them, including the ones your code catches with `try`.



The stack: total, called from line 9 of the module.

History, calls, performance and objects

- **History**: the last lines of your code that ran, with the time and the function. A click takes you to the line.
- **Calls**: every call to a function of your code and its return, indented by depth and with how long it took: the flow of the program.
- **Performance**: for every function, how many times it was called and its total, mean and maximum time (not counting debugger pauses), and the most executed lines. It updates by itself; **Reset** sets the counters to zero.
- **Objects**: how many live objects there are of each class of your program (and tkinter widgets): handy to see whether something is created too many times.

Tests

The **Tests** tab runs the `unittest` tests in the project's `tests` folder (`test_*.py` files). **Create a sample test** adds `tests/test_form1.py`: tests import your program with `from dms_app import Form1` (for a web app, `from dms_app import app` and its test client).

```
import unittest

from dms_app import Form1


class TestForm1(unittest.TestCase):
    def setUp(self):
        self.form = Form1()
        self.form.root.withdraw()

    def tearDown(self):
        self.form.root.destroy()

    def test_title(self):
        self.assertTrue(self.form.root.title())
```

Run tests shows every test in green, in red (click it to see its error) or skipped. **Debug tests** runs them under the debugger: they stop at the breakpoints in your code.

Chapter 5

Electron + React

Desktop apps with Electron and React: the same designer, with the code in JavaScript.

Creating an Electron project

When you create the app, choose **Electron + React**. The Designer, the inspector and the events are the same; the code language changes: **JavaScript**, in the `Form1.js` file. The palette only shows the components Electron already has.

> It needs **Node.js** (nodejs.org). The first time you run an Electron project, the studio installs Electron and React (about 100 MB) in its data folder; after that, every project shares them and starts at once.

The code: components with properties

Every event of the design is a method of the `Form1` class, which extends `FormDesign`. Every component is `this.Name`, with simple properties: when you change them, the window repaints (with React).

```
class Form1 extends FormDesign {
  form_create() {
    this.List1.items = ["Madrid", "Lima", "Bogotá"];
  }

  Button1_click() {
    this.Label1.text = "Hello, " + this.Entry1.text;
    this.Status.set("Greeted");
  }

  List1_select(event) {
    this.title = "Chosen: " + this.List1.selected;
  }
}
```

Component	Properties and methods
All	`text`, `enabled`, `visible`, `bg`, `fg`, `x`, `y`, `width`, `height`, `focus()`, `configure({...})`

Component	Properties and methods
Entry, Text, Password	`text` (also `value`), `readonly`
Check box, Radio button	`checked`
List, Combo box	`items`, `index`, `selected`, `selection` (several), `add(text)`, `clear()`, `remove(i)`
Data grid	`columns`, `rows` (lists or objects), `addRow(row)`, `index`, `selectedRow`, `clear()`
Scale, Progress bar	`value`, `maximum`; the indeterminate bar, `start()` and `stop()`
Page control	`page` (the active page)
HTML view	`html`, `setHtml(html, top)`; links and forms with `data-action` call its event
Status bar	`text`, `icon`, `set(text, icon)`
Timer	`interval`, `start()`, `stop()`
Open and Save dialogs	`await execute()` (the path or null), `readText(path)`, `writeText(path, text)`
The window (`this`)	`title`, `message(text)`, `ask(question)`, `notify(text)`, `after(ms, function)`, `close()`

The menu is the system's **native menu**: its event receives the chosen option, such as `"File/Open…"`. Node's `require` also works in the code (`fs`, `path`…).

Run, debug and export

- **Run (F9)** generates the app in `build/electron` and opens it in its own window. What `console.log` writes and the errors (with their `Form1.js` line) go to the **Output** tab.
- **Debug (Shift+F9)** opens it with Chromium's **developer tools**: your breakpoints become `debugger;` on the same line, and there you have variables, the stack, the console, stepping and performance.
- **Export the Electron app** creates a folder with its `package.json`: it opens on any computer with `npm install` and `npm start`.

> Not in Electron yet: the chart, the canvas, the ribbon, the API integration, the AI assistant, the reports, the tray icon and data connections. Nor “Ask the AI” or the Tests tab. They will come in future versions.

Chapter 6

Chrome extensions

An extension popup, designed the same way, with the code in JavaScript.

Creating an extension

When you create the app, choose **Chrome extension**. The design is the **popup** that opens when you press the extension icon (800 × 600 at most) and the code is the same JavaScript as Electron, with the same components (all but the menu).

The studio generates a complete **Manifest V3** extension in `build/extension`: `manifest.json`, `popup.html` and React packed inside (an extension cannot load outside code). It needs **Node.js** the first time, like Electron.

What an extension adds

From code	What it does
`await this.storage.get(key, fallback)`	Reads saved data (`chrome.storage`)
`await this.storage.set(key, value)`	Saves data (text, number, list or object)
`await this.currentTab()`	The active tab: `{ id, url, title }`
`this.openTab(url)`	Opens an address in a new tab
`await this.runInPage(function, ...args)`	Runs the function inside the open page and returns what it returns
`this.badge(text, color)`	Puts text on the icon badge

```
class Form1 extends FormDesign {
  async Count_click() {
    const text = await this.runInPage(() => document.body.innerText);
    const words = text.split(/\s+/).filter(Boolean).length;
    this.Words.text = words + " words";
    this.badge(String(words));
  }
}
```

The `runInPage` function runs in the page, not in the extension: it only sees what is in the page and what it receives in `args`. Chrome pages (`chrome://…`) cannot be read.

Test, load in Chrome and publish

- **Run (F9)** opens a **preview** of the popup: data is saved in the preview and the active tab is a sample one. **Debug** opens it with the developer tools.
- **Load in Chrome** prepares the extension, copies its folder and opens `chrome://extensions`: turn on **Developer mode**, press **Load unpacked** and choose that folder. After every change, save and press ↻ on the extension card.
- **Export the extension** creates its folder and a `.zip`, which is what you upload to the Chrome Web Store.

Chapter 7

Web with Next.js

Next.js web pages: the design is the page and the code is JavaScript with React.

A website with screens (like Flutter)

When creating the project, **Web (Next.js)** uses the same screen editor as Flutter: screens with their widget tree, properties, menu and the 20 templates. Each screen is a page of the website (the first is `/`; the others, `/its-name`), made in React with responsive HTML and CSS: content is centered on desktop and the menu turns from a side bar into a bottom bar on mobile. The logic goes in `lib/app.js`: an `App` class that calls `this.notify()` when it changes.

```
export class App extends DmsApp {
  tasks = [];
  newTask = "";

  add() {
    this.tasks.unshift({ title: this.newTask, done: false });
    this.newTask = "";
    this.notify();
  }
}
```

- Properties are written as in Flutter: `${…}` in texts, `=expression` (JavaScript) and the actions `go:Screen(item)`, `home:Screen`, `back` or code. `.isEmpty` and `.isNotEmpty` also work.
- In the editor, the preview shows the desktop browser or the phone (the palette button).
- **Run (F9)** starts `next dev` and opens the browser; saving reloads the page.
- Example: **Web shop**, with login, catalog, product page, cart and checkout.
- Earlier projects keep working as **Web (Next.js, form)**.

A website with a server

In **Web (Next.js)**, the code can have a part that runs on the server: everything after a `// @server` line is `lib/server.js`. Keys, calls to other sites' APIs and anything that must not reach the browser go there. The app calls each function it exports with `await server.name({ … })` (the studio generates the `app/api/[fn]/route.js` route) and gets back whatever it returns; if it returns a `Response`, it goes out as is (for downloads or data that isn't JSON).

```
import { DmsApp, server } from "./dms";

export class App extends DmsApp {
  price = null;

  async start() {
    this.price = await server.price({ currency: "EUR" });
    this.notify();
  }
}

// @server: what follows runs on the server.
export async function price({ currency }) {
  const r = await fetch("https://api.example.com/price?c=" + currency, {
    headers: { authorization: "Bearer " + process.env.API_KEY },
  });
  return r.json();
}
```

- The server part is another module: it doesn't see the app's variables (and the app doesn't see its own). Whatever both need goes in the arguments or in what it returns.
- Keys go in `.env.local` (in `build/nextjs` when running from the studio) or in Vercel's environment variables, and are read with `process.env`.
- `serverUrl("name", { … })` gives the address of a function, for your own `fetch`, a link or a download.
- Anything `lib/server.js` exports can be called from outside: check what arrives in the arguments.
- Examples: **Live World Map (Next.js)**, which merges on the server the planes, ships and trains from four public APIs and draws them on a Canvas, and **Internet Speed Test (Next.js)**, whose server returns a `Response` with random bytes to measure the download and counts the ones it receives for the upload, and **Flight Finder (Next.js)**, which keeps the Travelpayouts token on the server (`TRAVELPAYOUTS_TOKEN`).

A React (Vite) website

- **Web (React)** uses the same screen editor and the same logic (`src/lib/app.js`), but generates a single-page website with React and Vite: routes go after `#` (`#/catalogo`) and `npm run build` leaves static files in `dist/` for any hosting, with no server.
- **Run (F9)** starts Vite and opens the browser; saving reloads the page. Vite installs itself the first time (about 40 MB, shared).
- Example: **Web shop (React)**, the same shop as in Next.js and React Native.

A plain HTML website

- **Web (HTML)** uses the same screen editor and the same logic (`app.js`), but generates plain HTML, CSS and JavaScript: nothing to build, no Node.js and no dependencies. `index.html` loads `dms.js` (a small engine that draws the screen of the route after `#`), `app.js` and `screens.js`.
- The `App` class is the same as in Next.js and React; the `import` and `export` lines are ignored, so the logic can move between the three targets unchanged.
- **Run (F9)** serves the folder with the studio's own server and opens the browser; saving reloads the page.
- **Export** leaves the folder ready: open `index.html` or upload it to any hosting.
- Example: **Web shop (HTML)**, the same shop as in Next.js and React.
- **Canvas**: a screen editor component for drawing with code. **Draw** calls an `App` method with `ctx` (the browser's 2D context; in Flutter, the `Canvas`), `w` and `h` whenever the app notifies with `notify()`; **On tap** receives `x` and `y`. It fits the width and height of the screen with its **Aspect ratio**. It works in HTML, React, Vue, Next.js and Flutter; in React Native it shows a notice for now.
- Example: **Double-Nine Dominoes (web)**, a screen with a Canvas: the whole game goes in `app.js` and plays in the browser.

A Vue (Vite) website

- **Web (Vue)** uses the same screen editor and the same logic (`src/lib/app.js`), but each screen is a Vue component (`src/screens/Screen.vue`) with its template: `v-for` for lists, `v-if` for what is shown or hidden and `@click` for actions.
- `this.notify()` bumps a version the screens read, and Vue redraws them; the `App` class is the same as in Next.js, React and HTML.
- Routes go after `#` and `npm run build` leaves static files in `dist/` for any hosting.
- **Run (F9)** starts Vite and opens the browser; saving reloads the page. Vite and Vue install themselves the first time (shared with React).
- Example: **Web shop (Vue)**, the same shop as in the other websites.

A website with a Go server

- **Web (Go)** uses the same screen editor and the same logic (`app.js`), served by a Go server (`main.go`, standard library only) that provides a **REST API** with the data saved in JSON files (`data/<collection>.json`).
- From the logic: `await api.list("pedidos")`, `api.list("pedidos", { cliente: "Ana" })` (filter), `api.get(col, id)`, `api.add(col, row)` (returns the row with `id` and `created_at`), `api.update(col, id, changes)` and `api.remove(col, id)`.

- **Run (F9)** builds the server and opens the browser; saving reloads the page. The data goes in the project's `data` folder and is still there after a restart.
- **Export** leaves the Go project: `go run .` or `go build` (a single executable with the website inside). Needs Go (go.dev); on macOS, with Go older than 1.22, the studio links with the system linker.
- Example: **Web shop (Go)**: orders are saved on the server and the numbering continues after restarting it.

Creating a Next.js page

When you create the app, choose **Web (Next.js)**. The design is the page and the code is the same JavaScript as Electron (it runs in the browser), with the same components (all but the menu). The studio generates a Next.js project (app router) in `build/nextjs`.

- **Run (F9)** starts the Next.js development server and opens the browser when it is ready. Next.js, React and Electron are installed once and shared by every project.
- `this.storage` saves data in the browser; `this.currentTab()` returns the page itself and `this.runInPage(function)` runs it there.
- **Debug** turns your breakpoints into `debugger;`: open the browser tools (F12) and it will stop at them.
- **Export the Next.js project** creates a folder ready for `npm install` and `npm run dev`, or for deploying to Vercel.

Chapter 8

C++ with FLTK

Native desktop apps in C++: the same designer, compiled with FLTK.

Creating a C++ app

When you create the app, choose **C++ (FLTK)**. The code goes in `Form1.cpp`: the `Form1` class inherits from `FormDesign` and every event is a method with `override`. Every component is a pointer to an FLTK class with simple methods (`text`, `checked`, `items` …), without losing the FLTK API.

```
class Form1 : public FormDesign {
public:
  void Button1_click() override {
    Label1->text("Hello, " + Entry1->text());
    Status->text("Greeted");
  }

  void Scale1_change(double value) override {
    Bar1->value(value);
  }
};
```

Component	Methods
Label, Button, Status bar	`text()`, `text(s)`, `enabled(bool)`, `visible(bool)`, `bg(color)`, `fg(color)`
Entry, Password, Text	`text()`, `text(s)`
Check box, Radio button	`checked()`, `checked(bool)`
List, Combo box	`items({...})`, `add(s)`, `index()`, `index(i)`, `selected()`
Scale, Progress bar	`value()`, `value(v)`
Page control	`page()`, `page(i)`
Timer	`Timer1.start()`, `Timer1.stop()`
Open and Save dialogs	`OpenFile.execute()` (the path, or "" if cancelled)

Component	Methods
The window	`title(s)`, `message(text)`, `ask(question)`, `close()`; and `str(number)`, `readText(path)`, `writeText(path, text)`

Build, run and export

- **Run (F9)** compiles with the computer's C++ compiler and FLTK, and opens the app. Errors go to the Output tab with their `Form1.cpp` line.
- If FLTK or the compiler is missing, the studio says so and offers **Install** (on macOS, with Homebrew: `brew install fltk`).
- **Debug** compiles with debug information (`-g`) so you can open `build/cpp/app` with lldb or gdb.
- **Export the C++ project** creates a folder with the sources and a `Makefile`: `make run`.

Chapter 9

Pascal with Lazarus

Native desktop applications in Object Pascal, with the LCL components.

Create a Pascal application

When creating the application, choose **Pascal (Lazarus)**. The code goes in the `Form1.pas` unit: the `TForm1` class inherits from `TFormDesign` and each event is a method declared in the class with `override` and written in `implementation`. The studio adds both parts when you create an event. The components are the usual LCL ones.

```
type
  TForm1 = class(TFormDesign)
  public
    procedure Button1_click; override;
    procedure Scale1_change(Value: Double); override;
  end;

implementation

procedure TForm1.Button1_click;
begin
  Label1.Caption := 'Hello, ' + Entry1.Text;
  Status.SimpleText := 'Greeted';
end;

procedure TForm1.Scale1_change(Value: Double);
begin
  Bar1.Position := Round(Value);
end;
```

Component	LCL class and what is most used
Label	`TLabel` : `Caption` , `Font` , `Color`
Button	`TButton` (or `TDmsButton` when it has a color or is tall): `Caption` , `Enabled`
Entry, Password	`TEdit` : `Text` , `Clear` , `SetFocus`
Text	`TMemo` : `Text` , `Lines.Add` , `Lines.LoadFromFile`
Check box, Radio button	`TCheckBox` , `TRadioButton` : `Checked`

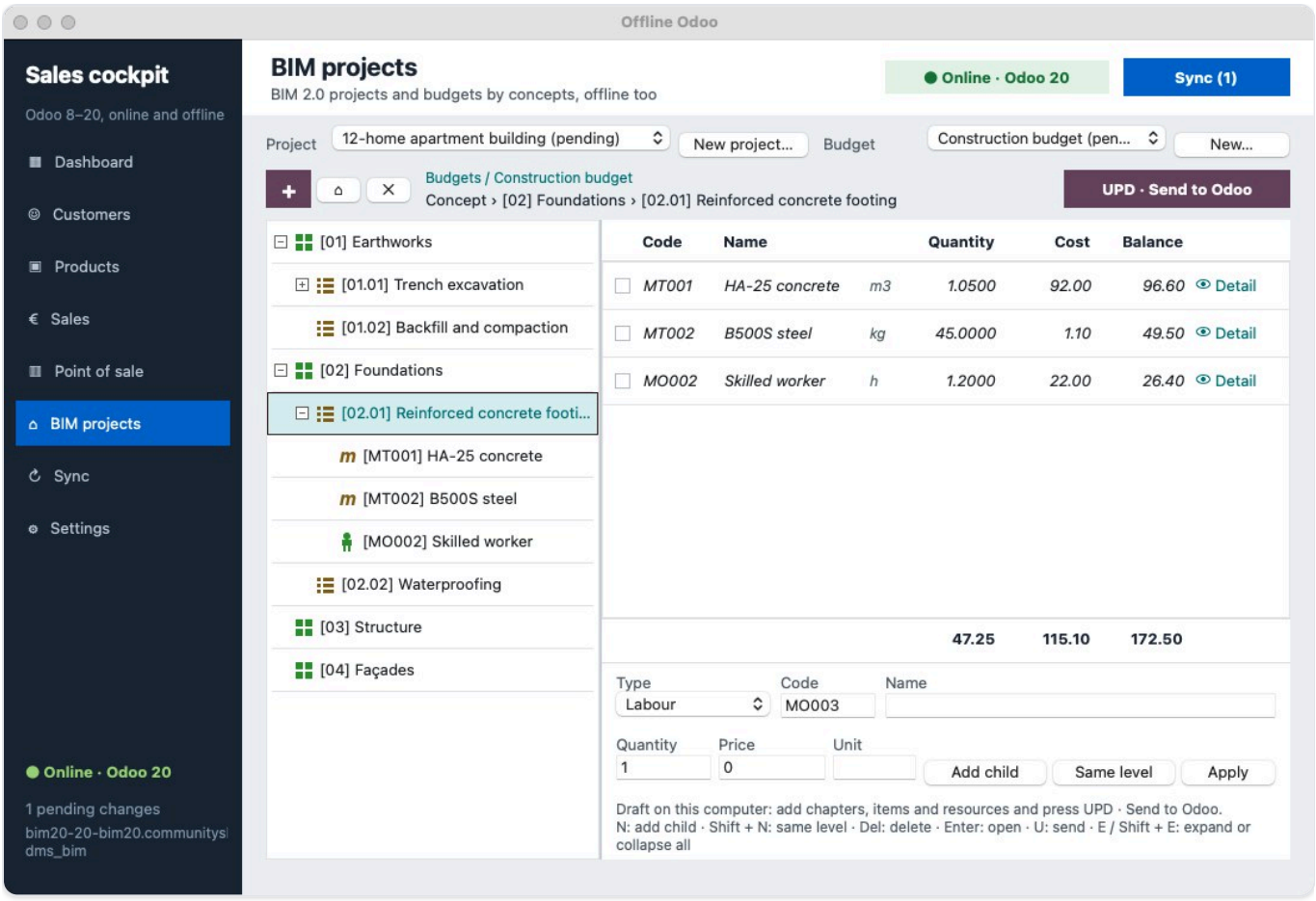
Component	LCL class and what is most used
List, Combo box	`TListBox`, `TComboBox`: `Items`, `ItemIndex`
Scale, Progress bar	`TTrackBar`, `TProgressBar`: `Position`, `Max`
Frame, Page control	`TPanel`, `TPageControl` (each page, `Pages1_page0`…)
Status bar	`TStatusBar`: `SimpleText`
Menu, Timer	`TMainMenu`, `TTimer`: `Enabled`, `Interval`
Open and Save dialogs	`TOpenDialog`, `TSaveDialog`: `Execute`, `FileName`
The window and the studio	`Caption`, `Close`, `ShowMessage(text)`, `Ask(question)`, `NumToStr(x)`, `ReadText`, `WriteText`, `LastKey` (in the key event)

Compile, run and export

- **Run (F9)** compiles with `lazbuild` and opens the application. Errors show in the Output tab with their `Form1.pas` line, such as `Form1.pas(22,3) Error: Identifier not found`.
- If Lazarus is missing, the studio says so and offers **Install** (on macOS, with Homebrew: `brew install fpc` and `brew install --cask lazarus`). On an Apple Silicon Mac it compiles for arm64.
- What `WriteLn` writes shows in the Output tab. To step through the code, open `build/pascal/app.lpi` in the Lazarus IDE.
- **Export the Lazarus project** creates a folder with `app.lpi` and the units: open it in Lazarus or build it with `lazbuild app.lpi`.

Example: offline Odoo and BIM projects

The **Offline Odoo (Pascal)** example is a native app that works with Odoo 8 to 20 over JSON-RPC, online and offline: dashboard, customers and vendors, products (which can also be created offline), sales and purchases with their steps (**Confirm**, **Invoice** and **Register payment**, which Odoo repeats when you sync), point of sale and, if the Odoo database has **BIM 2.0** (the `base_bim_2` module), the **BIM projects** page to make projects and budgets by concepts. It can be downloaded already built from the **Demos** page.



BIM projects, like the BIM 2.0 concept tree: the tree on the left; on the right, what is inside the selected item.

1. In **Settings**, type the Odoo address, database, user and password and press **Connect and download**. If Odoo has BIM 2.0, **BIM projects** appears in the menu, after **Point of sale**.
2. Choose an Odoo project or create one with **New project…** (name and, optionally, customer). Then create its budget with **New…**.
3. In the tree, with nothing selected (or with ⌂), add the chapters; select a chapter to add its items and an item to add its resources: labour, equipment, material, subcontract or function. The editor below suggests the type and code (01, 01.01, MO001…); type the name, quantity, price and unit and press **Add child**, **Same level** or **+**.
4. The table shows what is inside the selection with its **Quantity**, **Cost** and **Balance**, and the totals. Select a row to change it with **Apply**; **Detail**, a double click or Enter open it.
5. Press **UPD · Send to Odoo**: online it is uploaded at once; offline it waits in **Sync** and is sent when the network is back. Odoo recalculates the amounts (UPD) and the budget gets its PSU… code.

Key	In the tree or the table
N	Add a child to the selection
Shift + N	Add at the same level
Del	Delete (with everything inside)

Key	In the tree or the table
Enter	Open (go into the concept)
U	Send the budget to Odoo (UPD)
E / Shift + E	Expand or collapse the whole tree

Amounts are worked out as in BIM: a resource, quantity × price; an item, its quantity × the sum of its resources (or its price, if it has none); a chapter, the sum of what is inside. Chapters go on the first level, items inside a chapter and resources inside an item.

No duplicates: the project carries its DMS-… reference in its notes, the budget in its origin and each concept in its «id_import». Before creating anything it is looked up in Odoo, so resending duplicates nothing and an interrupted upload carries on where it stopped. Budgets already in Odoo are shown in the same tree (they are changed in Odoo). Tested with BIM 2.0 on Odoo 18 and 20.

Chapter 10

Mobile apps with Flutter

Screens in the style of FlutterFlow, logic in Dart and a built-in phone simulator.

Create a Flutter app

When creating the project, choose **Flutter (mobile)**. The **Design** tab is an editor in the style of FlutterFlow: on the left, the **screens** and the **widget tree** of the selected one; in the middle, the **palette** and the **phone** with the preview; on the right, the **properties** of the selected widget (or of the screen and the app).

- A widget from the palette is added inside the selected one if it is a column, a row or a list; otherwise, after it. Select it by tapping it on the phone or in the tree; **Up**, **Down**, **Duplicate** and **Delete** are above the tree.
- Widgets: column, row, list, card, container, padding, center, expanded, text, button, icon button, text field, list tile, check box, switch, chip, icon, image, progress, divider and spacer.
- **+** in Screens opens the gallery: a **blank** screen or one of the **20 most used templates** (sign in, sign up, reset password, welcome, home, list, detail, profile, edit profile, settings, notifications, messages, chat, search, cart, checkout, order confirmed, dashboard, contact and help), with its preview. Their buttons write to Output (`debugPrint`) until you bind them to App.
- Each screen has a name, a title, a top bar, a floating button and, if it is opened with something (a shipment, for example), the type it **receives**.

Logic and bindings

The **Code** tab is `lib/app.dart`: an `App` class with the app data and methods. The screens use it as `app` and redraw when it calls `notifyListeners()`. **Generated Dart** shows `lib/main.dart`, which the studio writes from the screens.

```
class App extends ChangeNotifier {
  final tasks = <Task>[];
  String newTask = '';

  void add() {
    tasks.insert(0, Task(newTask));
    newTask = '';
    notifyListeners();
  }
}
```

In the properties	Example
A text with values	`${item.title}` · `Hello, ${app.name}`
An expression (with =)	`=item.done` · `=app.color(item.status)`
An action	`app.add()` · `go:Detail(item)` · `app.create(); back`
A bound list	Items: `=app.tasks`; its first child repeats with `item`
A text field	Saves to: `app.newTask`
Visible if	`=app.tasks.isNotEmpty`
A conditional action	`if (app.signIn()) home:Catalog`
Bar buttons	`shopping_cart|go:Cart|${app.items}` (icon, action and badge)

Desktop

- Choose **Flutter (desktop)** when creating the project: the same editor, with the preview in a macOS window.
- **Menu** (app properties): one line per screen, `Screen|icon|text`. In a wide window it is a side bar (`NavigationRail`); in a narrow one, the bottom bar. It also works in mobile apps.
- **Run (F9)** opens the app in its native window (macOS, Windows or Linux), and saving reloads it.
- Example: **Inventory**, with a dashboard, products, a product page with stock and new products.

FalconDatabase and Firebase

In the app properties (with no widget selected), **Database → Connect to** chooses **FalconDatabase** (server, project reference and anon key) or **Firebase** (project ID and web key). The studio generates `lib/db.dart` with the same shape for both, and `App` uses it with `import 'db.dart';`.

```
Future<bool> signIn() async {
  try {
    await db.signIn(email, password);
    notes = await db.table('notes').list(order: 'id.desc');
    notifyListeners();
    return true;
  } on DbError catch (e) {
    error = e.message;
    notifyListeners();
    return false;
  }
}
```

- `db.signIn(email, password)`, `db.signUp(…)`, `db.signOut()` and `db.user`: the session.
- `db.table('t').list(where: {…}, order: 'field.desc', limit: 20)`, `get(id)`, `insert({…})`, `update(id, {…})` and `delete(id)`. In `where`, a value or an operator such as `gte.10`.
- Actions can wait: `if (await app.signIn()) home:Notes`.
- Only the public key (anon or the web key) goes inside the app; access rules are set by the database.
- Example: **Cloud notes**, with login and the FalconDatabase `notas` table.

The simulator and export

- **Run (F9)** compiles the app with Flutter for the web and opens it in the **studio simulator**: a phone frame (iPhone, Pixel, Galaxy or iPad mini) you can rotate. No Xcode or Android needed.
- When you **save**, the app is recompiled and the simulator reloads by itself.
- What `print` or `debugPrint` writes shows in the Output tab, and errors with their `lib/app.dart` line.
- pub.dev packages (for example, `shared_preferences: ^2.3.0`) are added in the app properties.
- **Export the Flutter project** leaves `pubspec.yaml`, `lib/main.dart` and `lib/app.dart`: with `flutter create .` and `flutter run` it opens on the phone, in Android Studio or in VS Code.
- Examples: **Shop with login** (login, catalog with search and categories, product page, cart with quantities and checkout), **Shipments** (summary by status, filtered list, detail with tracking and new shipments) and **Tasks** (add, check, filter and delete, saved on the phone).

Chapter 11

Mobile apps with React Native

The screen editor with native React Native (Expo) components and the logic in JavaScript.

Create a React Native app

When creating the project, choose **React Native (mobile)**: the same screen editor as Flutter (screens, widget tree, properties, menu and the 20 templates). Each screen is generated with React Native components (`View`, `Text`, `Pressable`, `TextInput`, `ScrollView`, `Switch`…) and the logic goes in `lib/app.js`, the same `App` class as in the Next.js website (with `this.notify()`).

- **Run (F9)** starts Expo and opens the app in the studio phone simulator; saving reloads it. The first time it installs React Native and Expo (about 340 MB, shared by all projects).
- The Output shows the `exp://…` address to open the app on your phone with **Expo Go** (on the same network).
- **Export the React Native (Expo) project** leaves `package.json`, `App.js`, `screens/` and `lib/`: `npm install` and `npx expo start`.
- Example: **Shop (React Native)**, with login, catalog, product page, cart and checkout.

Chapter 12

Console applications

Programs without a window in Python, C, C++ or Pascal, with input and output in the Output tab.

Create a console program

When creating the project, choose **Console** and the language: **Python**, **C**, **C++** or **Pascal**. There is no design: the project is a single file (`main.py`, `main.c`, `main.cpp` or `main.pas`) edited in the **Code** tab, with highlighting, autocomplete and help (F1).

```
name = input("What is your name? ")
print(f"Hello, {name}.")
```

- **Run (F9)** compiles if needed and starts the program. What it writes (`print`, `printf`, `std::cout`, `WriteLn`) shows in the **Output** tab.
- When the program asks for something (`input`, `scanf`, `std::cin`, `ReadLn`), type the answer in the field at the bottom of Output and press **Enter**: it shows after the question, as in a terminal.
- C and C++ are compiled with the computer's compiler; Pascal with Free Pascal (`fpc`), without Lazarus. Errors show with their line, such as `main.c:2:3: error: …`.
- **Export the program** saves the file as it is.

Examples

- **Animal management (console)**: in Python, with a menu to register and remove animals and keep track of their vaccines (next doses and those due in the next 30 days). It saves the data in `animales.json`, in the project folder.
- **Guess the number**, in Python, C, C++ and Pascal: the program thinks of a number between 1 and 100 and you guess it with the hints “Higher” and “Lower”.

Chapter 13

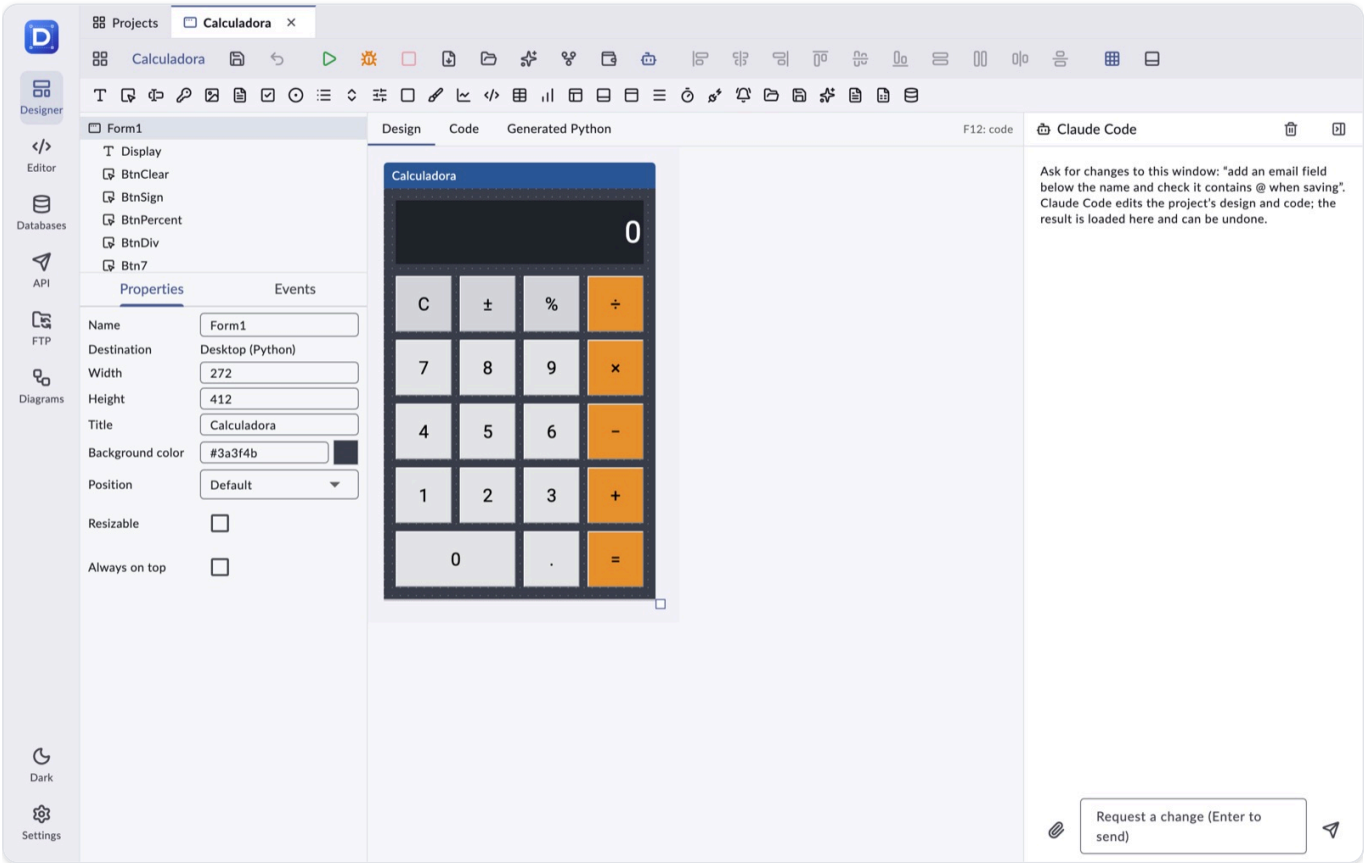
AI in the Designer

Chat with Claude Code and “Ask the AI”.

Chat with Claude Code

The **Chat with Claude Code** button (the robot on the toolbar) opens a panel on the right. Describe the change and press Enter (Shift+Enter adds a new line).

- It uses the Claude Code installed on your computer and your session: no API key needed.
- Before each message the studio saves the project. Claude Code reads and edits the design (`project.dmsproj`) and the window's code, without a terminal.
- When it finishes, the new design and code are loaded into the Designer as a single step that can be undone.
- The conversation continues between messages; the trash button starts a new one.
- The 📎 clip attaches images (a sketch, a screenshot) or text files: Claude Code reads them without them entering the project.



The Claude Code panel, to the right of the form.

Ask the AI

Ask the AI uses the provider set in **Settings** (Claude by default) and has three modes:

- **Switch** changes the current window (“Add a phone field and align the buttons”).
- **New window** designs a window from scratch; you can **Attach a sketch** (an image).
- **Code of** a component: with a component selected, it writes the method for its main event.

The answer arrives as an **AI proposal**: you review it before applying it and, once applied, it is undone in one step.

AI budget

The wallet button (**AI budget**) shows what the project has spent on Claude: by tool (Claude Code, Claude: implement, Ask the AI, chat), by model and by day, with the latest calls and their tokens.

- You can set a **Cap (US$)**, **Monthly** or **Total**. It warns at 80% and, with **Block when used up**, the AI stops answering in that project until you raise it.
- Costs use Anthropic API pricing. With a Claude subscription, Claude Code's cost is its API equivalent, not what you are charged.

GitHub

- The Designer's **GitHub** button connects your account with a token (kept only on this computer).
- **Publish on GitHub** creates the repository (private or public), makes the first commit and pushes it.
- Once published: it lists the changes, asks for the **Commit message** and **Commit and push**; also **Pull changes** and **Open on GitHub**.

Chapter 14

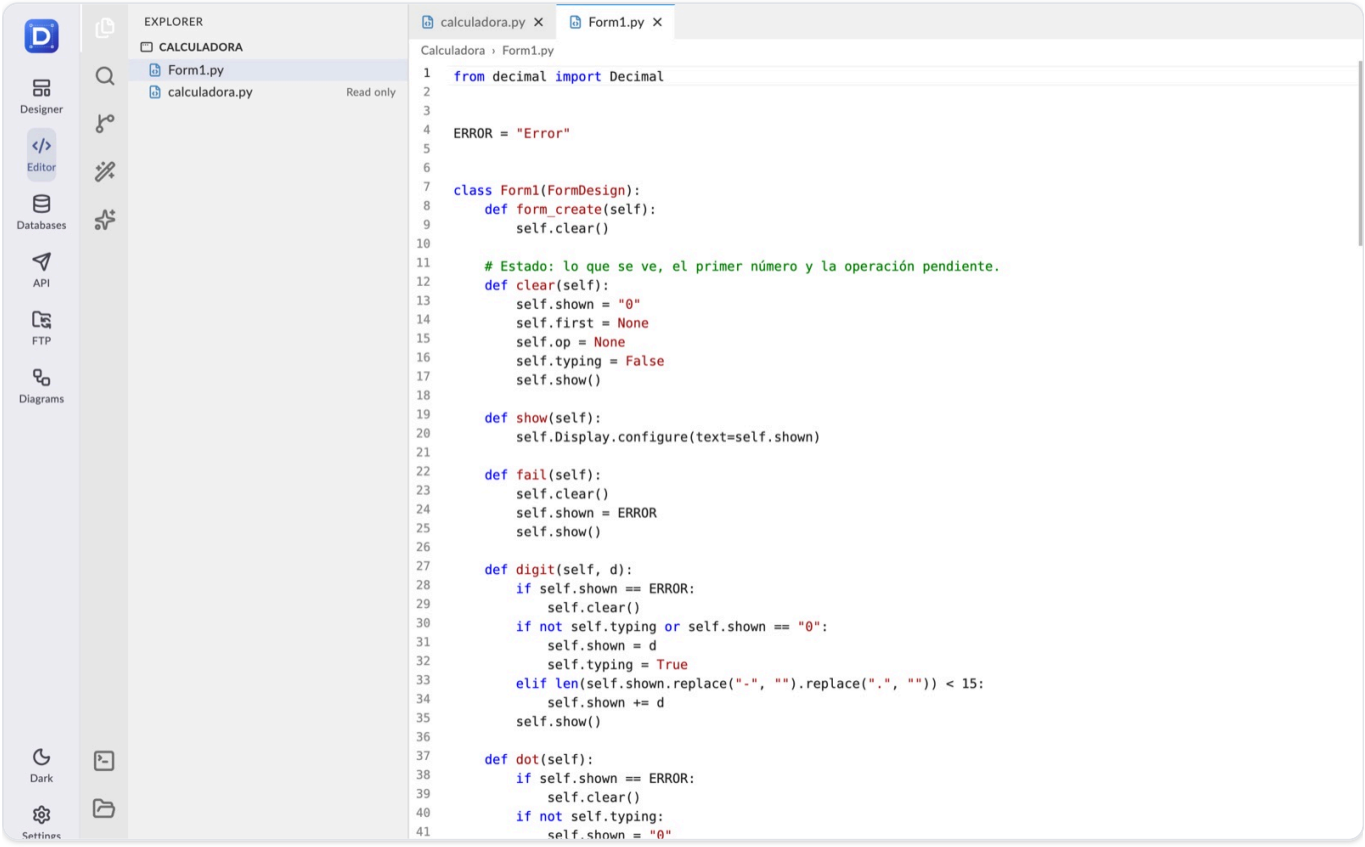
The Editor

Code editor with explorer, search, terminal, git and AI.

Opening a folder

With no folder open you get **Open folder...**, **Open DMarlon Studio** (the studio's documents folder), **Clone repository...** and the list of **Recent** folders.

If a project is open in the Designer, the Editor shows its code and its generated Python as **Project open in the Interface designer**: what you type here is the same text as in the Code tab.



The Editor: activity bar, explorer and tabs.

Explorer and tabs

- The activity bar holds **Explorer**, **Search**, **Source control**, **Claude: implement** and **AI chat**, with the terminal and **Open folder** at the bottom.
- The explorer offers **New file**, **New folder**, **Rename**, **Delete**, **Copy path**, **Open in terminal** and **Collapse all**.

- Tabs have syntax highlighting for dozens of languages and line numbers. ⌘S / Ctrl+S saves; closing something unsaved asks first.
- Anything unsaved is kept when you switch modules.

Find and replace

- **Find in file** with **Replace** and **Replace all**.
- The **Search** panel goes through every file in the folder, with **Match case** and **Regular expression**; a click opens the file at that line.

Terminal

⌘J / Ctrl+J shows or hides the terminal, which runs commands in the open folder (`git status`, `python3 app.py`, `npm install` …). The up and down arrows go through the history and **Stop** ends the running command.

Source control (git)

- **Changes**: modified files with their diff; type the **Commit message** and use **Commit all** or **Commit and push**. Also **Amend last commit** and **Undo last commit**.
- **Push**, **Pull** and **Check the remote for changes (fetch)**, with what is pending each way.
- **History** of the branch or of **All branches**: view the commit, **Revert this commit**, **Apply to the current branch (cherry-pick)**, create a branch or a tag from it and **Reset the branch to this commit…**.
- Local and remote **Branches**: create, switch, **Merge**, rename and delete (also on the remote).
- **Stash changes** to switch branches and bring them back later.
- **Remotes…**, **Initialize repository**, **Clone repository…** and **Publish to GitHub…** (public or private) with your **GitHub account** (a token with the “repo” scope).

Claude: implement

Claude: implement asks Claude for a change through the Anthropic API (Claude Opus 5.5 by default). Claude explores the open folder (lists, reads and searches files) and proposes changes to one or more files.

- Nothing is written until you apply it: the **Proposed changes** list shows each file with its added and removed lines; a click opens the diff with **Apply** and **Discard**.

- **Apply all** writes every change; if a file changed on disk while Claude was working, you are warned instead of it being overwritten.
- It needs the Anthropic key in **Settings → Artificial intelligence**.
- The 📎 clip attaches images and text or code files to the request; the wallet button opens the folder's **AI budget**.

AI chat

The **AI chat** answers about the open file with the provider from **Settings**. If it proposes the whole file, **Apply to the file** puts it in the tab (unsaved) and **Copy the code** copies it.

In all three chats Enter sends, Shift+Enter adds a line break and the 📎 clip attaches images (PNG, JPG, GIF or WebP up to 5 MB) and text files (up to 300 KB).

Chapter 15

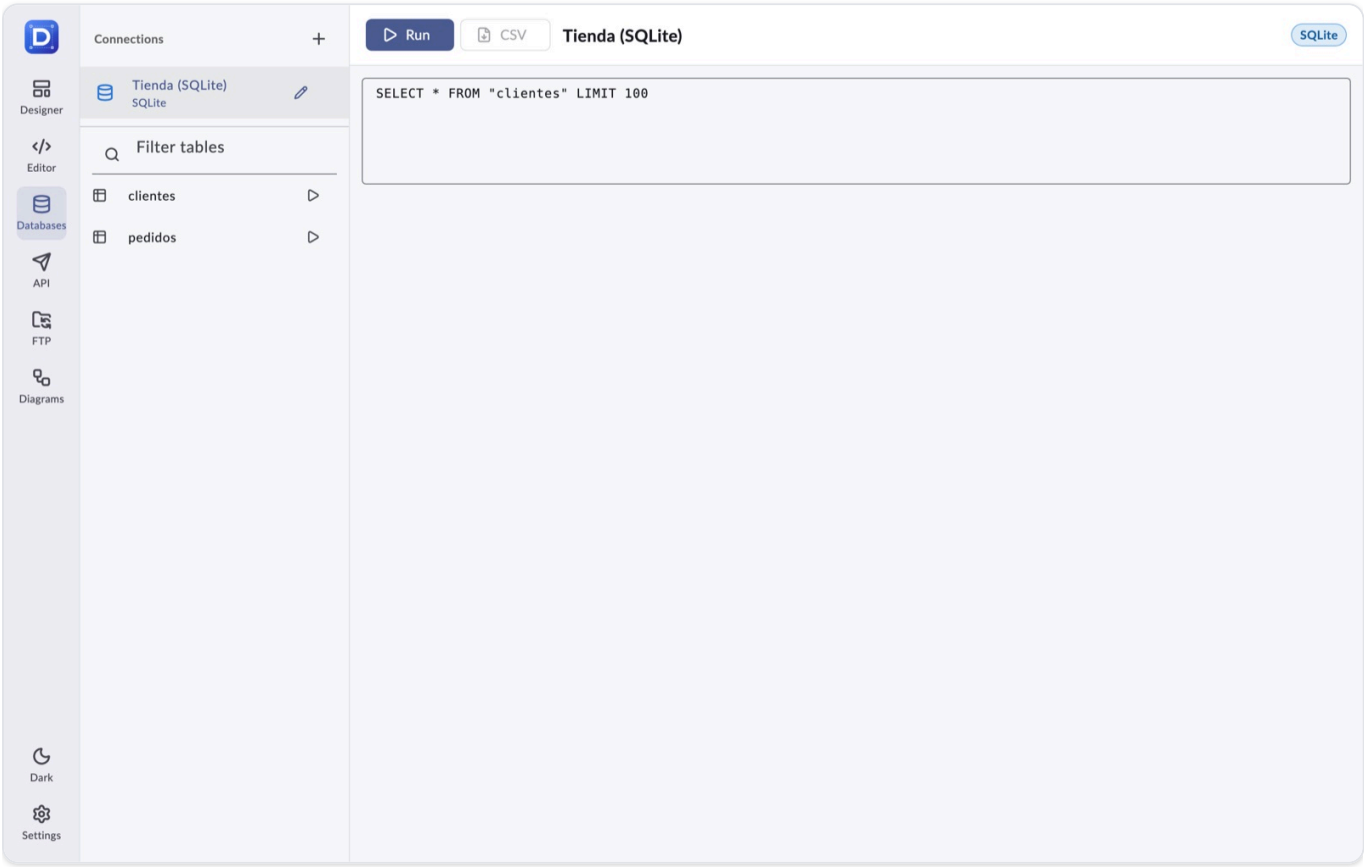
Databases

Connections, tables, SQL queries and CSV export.

Connections

+ adds a SQLite, PostgreSQL, MySQL / MariaDB, MongoDB, FalconDatabase or Firebase connection, with server, port, database, user, password and **Encrypted connection (SSL)** depending on the type. **Test connection** checks it can be reached.

The connections saved here are the ones the Designer's windows can use through the **Data connection** component.



A SQLite database with its tables and the query editor.

Tables and queries

- The table list can be narrowed with **Filter tables**; each table opens a sample query.
- **Run** (⌘↵) executes the query in the editor and shows the result in a grid.
- **Scripts**: with several queries in the editor (separated by `;` in SQL, or one JSON after another in MongoDB, FalconDatabase and Firebase) they all run in order and each leaves its result in a tab (**1 · 5**

rows, **2 · Done**…). If one fails, it stops there and shows the error with the results of the previous ones.

- If there is **selected** text, only the selection runs.
- SQL scripts ignore the `;` inside strings, comments, PostgreSQL `$$ … $$` blocks and the `BEGIN … END` body of a SQLite trigger; in MySQL, `DELIMITER //` changes the separator for procedures, as in MySQL Workbench. JSON accepts `//` comments.
- Anything that changes data asks for confirmation first (once per script).
- **CSV** saves the full result to a file.

FalconDatabase and Firebase

It also connects to cloud databases through their API: **FalconDatabase** (PostgreSQL tables) and **Firebase** (Cloud Firestore). Their queries are a JSON, as in MongoDB.

Type	What it needs	Sample query
FalconDatabase	Server (falcondatabase.com by default), the project reference and its key (`fdb_anon_…` or `fdb_service_…`); email and password if the tables require sign-in	`{"table": "products", "where": {"price": "gte.10"}, "order": "id.desc", "limit": 100}`
Firebase	The project ID, the web key (`apiKey`) and, if the rules require sign-in, a Firebase Authentication user; or a local emulator	`{"collection": "products", "where": {"price": {"gte": 10}}, "order": "price desc"}`

- To change data: `"insert": {…}` or a list, `"update": {"id": …, "set": {…}}` and `"delete": id`. In both, `update` and `delete` also accept `"where": {…}` instead of the id: they change or delete every row that matches. You are asked to confirm first.
- FalconDatabase: `"select": "id,name"`, `"order"`, `"limit"` and `"offset"`; a listing tells how many rows there are in total.
- Firebase: subcollections as a path (`"collection": "orders/o1/lines"`), `"group": true` to search every subcollection with that name, `"id"` for one document, `"select"`, `"offset"`, ordering by several fields (`"price desc, name"`) and the aggregations `"count": true`, `"sum"` and `"avg"`.
- MongoDB: besides `find` (with `projection`, `sort`, `limit` and `skip`), `aggregate`, `insert`, `update` and `delete`, also `"count": true`, `"distinct": "field"`, `"createIndex": {…}` (with `"unique"`), `"indexes": true` and `"drop": true`.
- FalconDatabase filters use its operators: `eq`, `neq`, `gt`, `gte`, `lt`, `lte`, `like`, `ilike`, `is` and `in`. Firebase: `eq`, `neq`, `gt`, `gte`, `lt`, `lte`, `in` and `contains`.
- In Firebase every row has `_id`, the document identifier. Operators also include `nin` and `containsAny`.

- The website has a course for each database (SQLite, PostgreSQL, MySQL, MongoDB, FalconDatabase and Firebase), with scripts that run here as they are: **Courses** in the menu.
- For now they are for the Databases module: the Designer's **Data connection** component uses SQLite, PostgreSQL and MySQL.

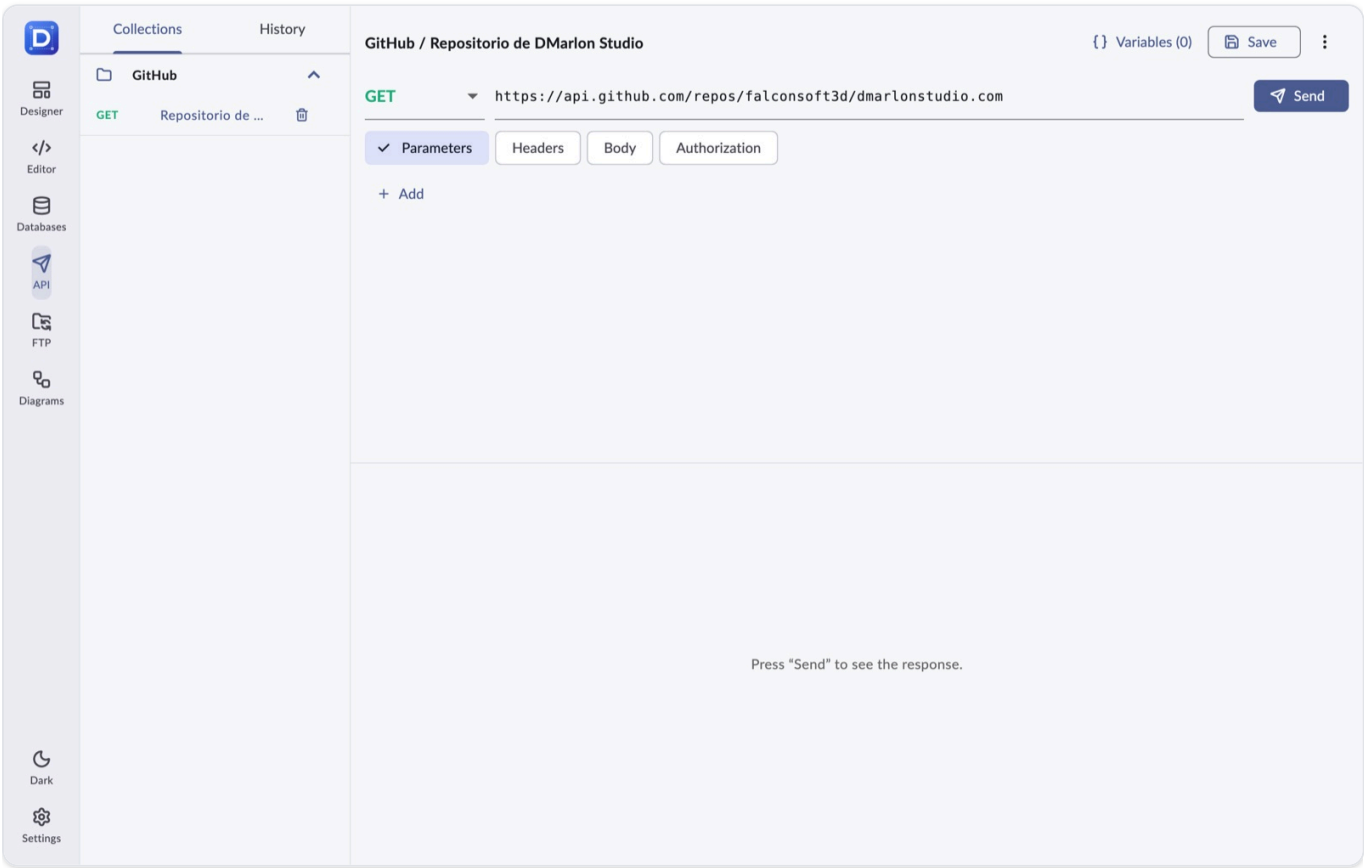
Chapter 16

API client

HTTP requests with collections, variables and history.

Requests

- Pick the method (GET, POST, PUT, PATCH, DELETE…), type the URL and click **Send**.
- **Parameters**, **Headers**, **Body** (JSON, form or text) and **Authorization** (none, Bearer or Basic) tabs.
- The response shows the status code, the time, the headers and the body.
- **Copy as curl** gets the request ready for the terminal.



A request saved in the GitHub collection.

Collections, variables and history

- **Save** stores the request in a **Collection**; **Save as new…** makes a copy.
- **Environment variables…** defines values such as `base_url` or `token`, used in the URL, headers and body as `{{base_url}}`.
- The **History** keeps the last 100 requests sent; **Clear history** empties it.

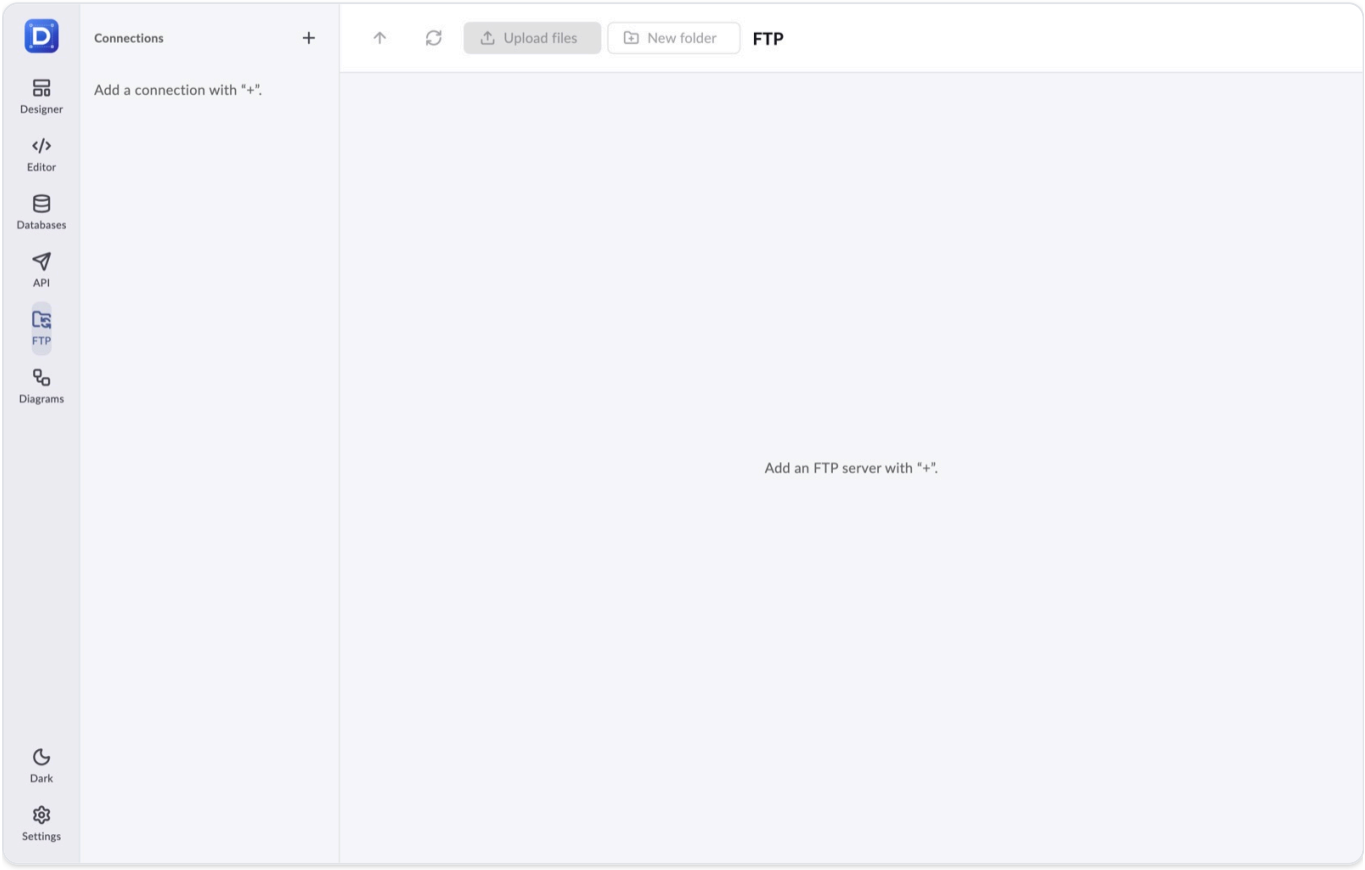
Chapter 17

FTP

FTP, FTPES and FTPS servers.

Servers and files

- **+** adds a server with its **Security**: **FTP (unencrypted)**, **FTPES (explicit TLS)** or **FTPS (implicit TLS)**.
- Choose a server to browse its folders; **Parent folder** goes up a level.
- **Upload files**, **Download**, **New folder**, **Rename…** and **Delete** (folders, with their contents).



The FTP client, ready to add a server.

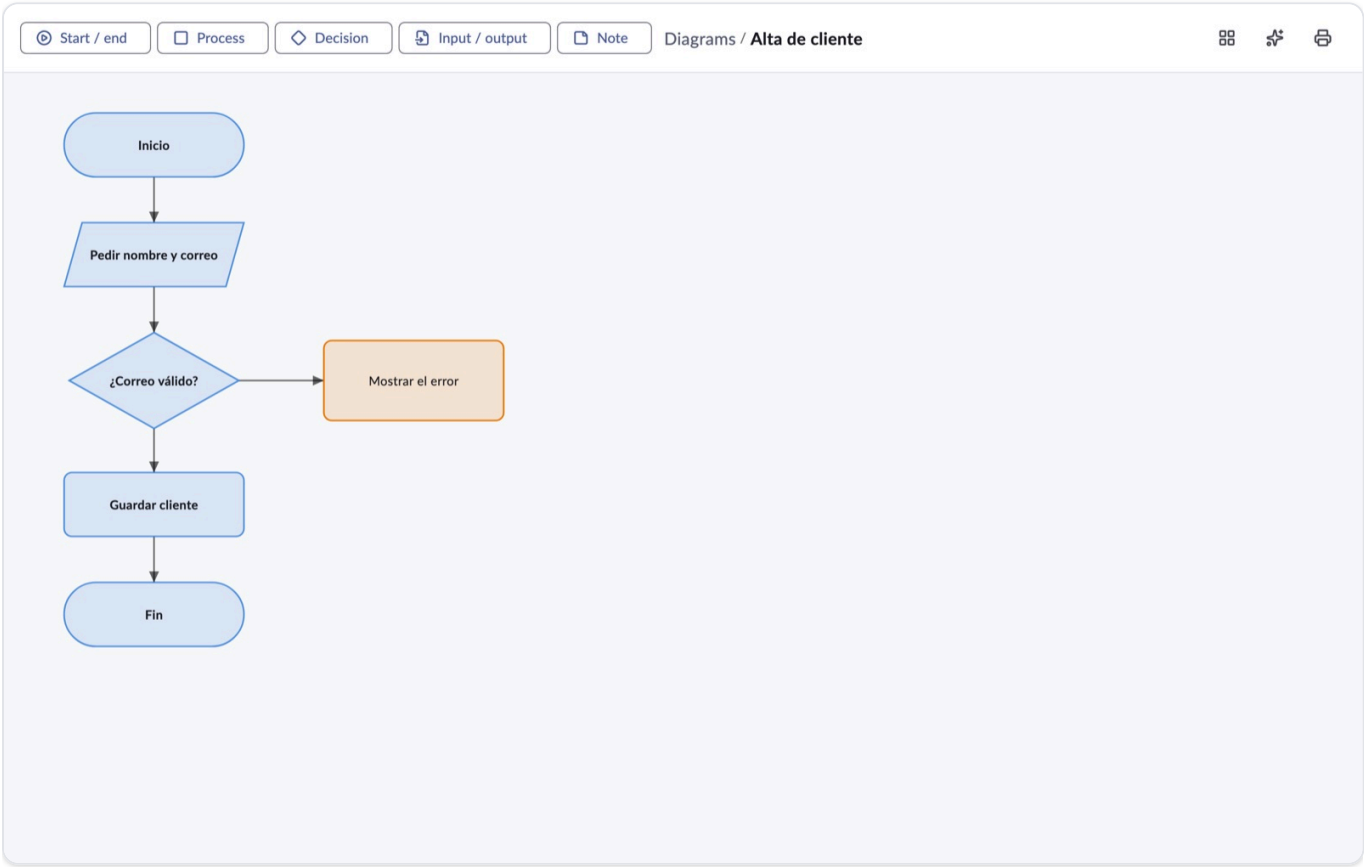
Chapter 18

Diagrams

Flowcharts and database diagrams, with AI and PDF.

Flowcharts and database diagrams

- **Flowchart**: **Start / end**, **Process**, **Decision**, **Input / output** and **Note** shapes. Press → next to a shape to connect it to another; connections can have a label (e.g. “Yes”).
- **Database**: tables with their columns, **Primary key** and foreign keys, and the relations between tables. **Diagram SQL** shows the SQL that creates it.
- **From a database** draws the diagram of a saved connection (optionally, only the tables starting with a prefix).
- **With AI** creates the diagram from a description and **Change with AI** modifies it.
- **Arrange** lays the shapes out again and **Export to PDF** saves it for printing or sharing.



A flowchart for registering a customer.

Chapter 19

Settings

Language, appearance and artificial intelligence.

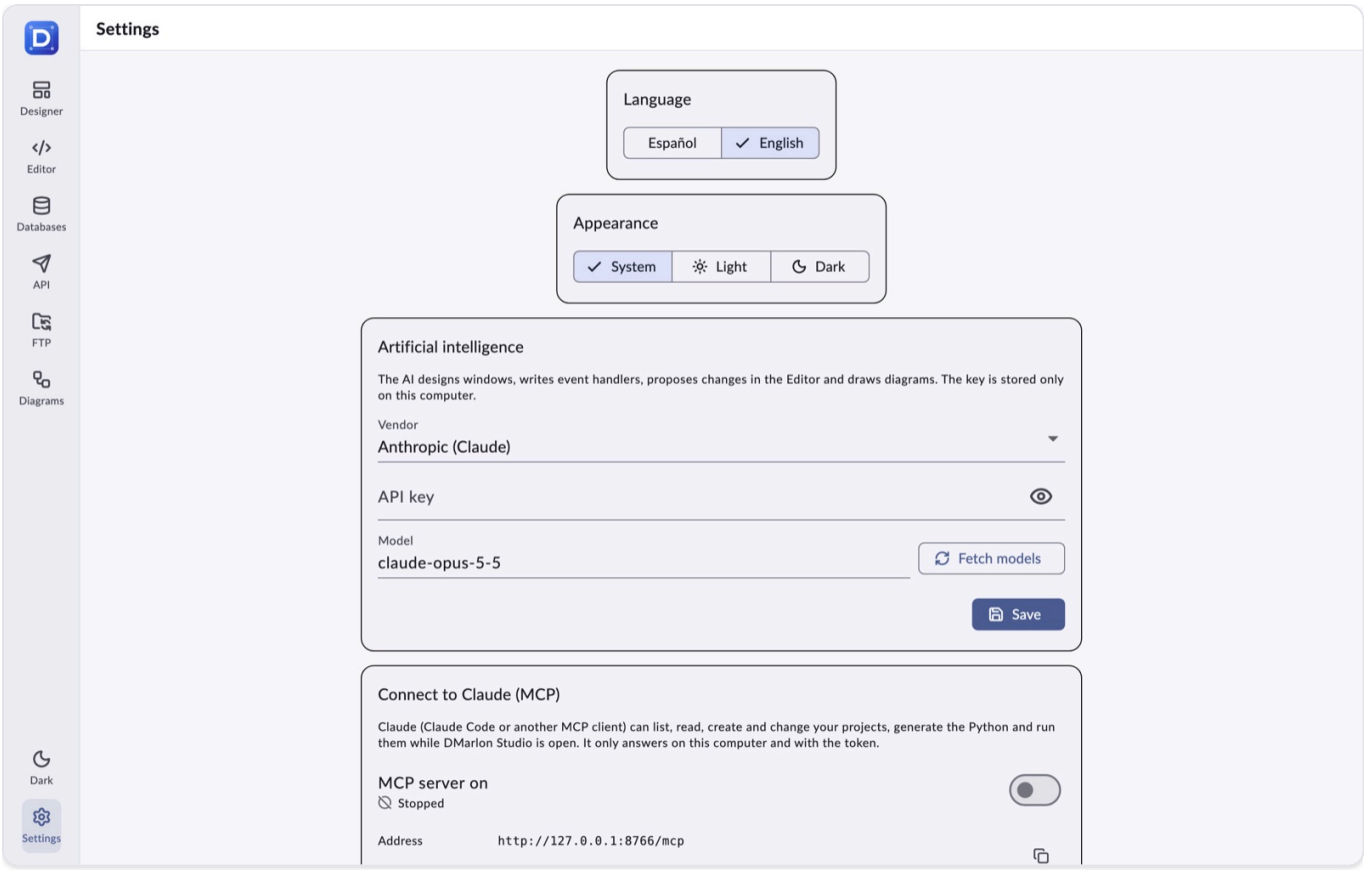
Connect to Claude (MCP)

- In **Settings → Connect to Claude (MCP)** you turn on the **MCP server**: while DMarlon Studio is open, Claude (Claude Code or another MCP client) can work with your projects.
- Tools: `list_projects`, `get_project`, `list_components`, `create_project`, `add_component`, `update_component`, `remove_component`, `update_code`, `generate_python`, `run_project`, `get_output` and `stop_project`.
- It only answers on this computer (`127.0.0.1`) and with the **Token**. **Add to Claude Code** registers it in one click; you can also copy the `claude mcp add` command shown.
- If a project is open in the Designer with no pending changes, it reloads by itself with what Claude does; with unsaved changes, Claude is told to save them first.

Artificial intelligence

Choose the provider (**Vendor**): Anthropic (Claude), OpenAI, Gemini, xAI, DeepSeek, Mistral, Groq, Cohere, OpenRouter, Perplexity, Ollama or a compatible custom API. Type the **API key** and the **Model**; **Fetch models** lists those on your account. Claude (`claude-opus-5-5`) is the default provider.

The key is stored on this computer only. Each provider remembers its key and model when you switch between them.



Settings: language, appearance, artificial intelligence and version.

Language, appearance and version

- **Language**: Español or English, instantly.
- **Appearance**: System, Light or Dark.
- **About** shows the installed version.

Chapter 20

Keyboard shortcuts

Every Designer and Editor shortcut.

Designer

Shortcut	Action
F9	Run (when debugging and stopped: continue)
Shift+F9	Debug
F5	Set or remove a breakpoint on the cursor line
F7 / F8 / Shift+F8	Step into / step over / step out
F4	Run to cursor
Ctrl+F2	Stop
F12	Switch between design and code
Ctrl+Space	Autocomplete in the code
F1 / right-click → Help	Help on the word at the cursor
⌘F / Ctrl+F	Find (and replace) in the code
⌘S / Ctrl+S	Save the project
⌘Z / Ctrl+Z	Undo
⌘C, ⌘X, ⌘V (Ctrl on Windows)	Copy, cut and paste components
Delete / Backspace	Delete the selection
Arrows	Move one grid step (with Alt, one pixel)
Shift + arrows	Resize
Double-click on the palette	Add the component
Double-click on a component	Open its main event
Enter in a chat	Send (Shift+Enter: new line)

Editor

Shortcut	Action
⌘S / Ctrl+S	Save the file
⌘J / Ctrl+J	Show or hide the terminal
↑ / ↓ in the terminal	Go through the command history
Enter in a chat	Send (Shift+Enter: new line)

Chapter 21

Troubleshooting

The most common messages and how to solve them.

Python not found, or no tkinter

- **Python was not found**: install it (python.org or Homebrew) and click **Look for Python again** on the project list.
- **Found … but it does not include tkinter**: on macOS, `brew install python-tk`; on Linux, `sudo apt install python3-tk`; on Windows, reinstall Python ticking **tcl/tk and IDLE**.
- **Tk 8.5 is old and does not honor the colors of the design**: that is the Python bundled with macOS. Install one from python.org or Homebrew.

Flask is missing

Web apps need Flask in the same Python. The **Flask is missing** message shows the exact command, for example `python3 -m pip install flask`.

The app won't open (unsigned)

- **macOS**: the first time, right-click DMarlon Studio → **Open** → **Open**.
- **Windows**: on “Windows protected your PC”, **More info** → **Run anyway**.

AttributeError: 'Form1' object has no attribute 'root'

This happened when the window's code defined its own `__init__` without calling `super().__init__()`: that method replaced the one that creates the window. Since version 1.0.5 the window and its components are created before your `__init__`, and the **On open** event runs after it.

```
class Form1(FormDesign):
    def __init__(self):
        super().__init__()   # first
        self.current = "0"

    # Even better: the On open event
    def form_create(self):
        self.current = "0"
```

Claude Code was not found

Install Claude Code from claude.com/claude-code and run `claude` once in a terminal to sign in. The studio looks for it on the PATH and in `~/.local/bin`.

Chapter 22

Roadmap

What comes next.

Targets

You choose the target when creating a project; it cannot be changed later:

Target	Status
Python + tkinter (desktop)	Available
Python + Flask (web)	Available

Richer projects

- Several forms per project, with a **Project** node in the tree.
- The application icon, set in the project.
- Building executables (PyInstaller for Python).